**CAN THO UNIVERSITY**

**COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY**


**Project – Fundamental Topics**

# A MOBILE PERSONAL FINANCE MANAGEMENT APPLICATION SUPPORTED BY A LARGE LANGUAGE MODEL — PERFIN

| | |
|---|---|
| **Major:** | Software Engineering |
| **Cohort:** | 49 |
| **Course class:** | CT239H M01 |
| **Advisor:** | Dr. Phan Phuong Lan |
| **Student:** | Nguyen Thanh Trong |
| **Student ID:** | B2305615 |

**Semester 3, academic year 2025–2026**

Can Tho, 2026

## ABSTRACT

**Context:** Manual entry makes personal-finance histories incomplete or inconsistent, while an LLM must not become the source of ledger truth. **Objective:** This project builds PERFIN, a mobile prototype for natural transaction entry, deterministic analysis and reviewable writes. **Method:** Text, receipt images and speech become typed drafts; PostgreSQL is updated only after validation and confirmation. PaddleOCR and PhoWhisper run locally on the backend host, while Gemini is restricted to language extraction and fact narration. Balances, trends, anomalies, cash flow, budgets and goals are computed by deterministic services. **Results:** The current CSV profile contains 5,328 source rows and zero validation errors. With Redis/jobs disabled and the local parser selected, 46/46 backend test files and 31/31 parser cases pass. Eight manual functional cases are specified with provisional "Pass" statuses in the draft and require author verification before submission; UAT and live services remain unmeasured. **Significance:** The design keeps data and algorithms authoritative, exposes preview/confirmation and fails closed when local media runtimes are unavailable. The hybrid strategy preserves evidence of core correctness without treating author-run checks as usability or production-performance claims.



**Keywords:** personal finance management, large language model, data analysis, PostgreSQL, verifiable systems.

# Contents

# List of Tables

# List of Figures

## LIST OF ABBREVIATIONS

| No. | Abbreviation | Meaning |
|---|---|---|
| 1 | AI | Artificial Intelligence |
| 2 | API | Application Programming Interface |
| 3 | ASR | Automatic Speech Recognition |
| 4 | BOM | Byte Order Mark |
| 5 | CER | Character Error Rate |
| 6 | CRUD | Create, Read, Update, Delete |
| 7 | CSV | Comma-Separated Values |
| 8 | DB | Database |
| 9 | ERD | Entity–Relationship Diagram |
| 10 | FK | Foreign Key |
| 11 | FR | Functional Requirement |
| 12 | HTML | HyperText Markup Language |
| 13 | HTTP | HyperText Transfer Protocol |
| 14 | IEEE | Institute of Electrical and Electronics Engineers |
| 15 | IQR | Interquartile Range |
| 16 | JSON | JavaScript Object Notation |
| 17 | KV | Key–Value |
| 18 | LLM | Large Language Model |
| 19 | NFR | Non-Functional Requirement |
| 20 | OCR | Optical Character Recognition |
| 21 | OLS | Ordinary Least Squares |
| 22 | PDF | Portable Document Format |
| 23 | PERFIN | Personal Finance — the name of the prototype presented in this report |
| 24 | PK | Primary Key |
| 25 | REST | Representational State Transfer |
| 26 | SQL | Structured Query Language |
| 27 | STT | Speech-to-Text |
| 28 | TC | Test Case — a manual functional test-case identifier |
| 29 | TTL | Time To Live |
| 30 | UAT | User Acceptance Testing |
| 31 | UI | User Interface |

| No. | Abbreviation | Meaning |
|---|---|---|
| 32 | UML | Unified Modeling Language |
| 33 | VND | Vietnamese Dong |
| 34 | WER | Word Error Rate |

## STATUS AND RESULT CONVENTIONS

This report clearly distinguishes implemented components, measured results and behavior that exists only at the design level. This convention prevents expectations or smoke tests from being presented as completed quality evidence.

Convention for levels of information confidence in the report

| Label | Meaning | Usage |
|---|---|---|
| Implemented | A corresponding component exists in source code, migrations or tests. | Proves existence; does not automatically prove quality. |
| Measured | A command, timestamp and reproducible run result exist. | May be used to report figures in the results section. |
| Target | A desired threshold used as an acceptance criterion. | Must not be presented as an achieved result. |
| Design target | Expected behavior after in-scope fixes are completed. | Must be tested before being converted to “measured”. |
| Out of scope | Not a goal of the current project. | Mentioned only under limitations or future work. |

# CHAPTER 1: INTRODUCTION

## 1.1. PROBLEM STATEMENT

Personal finance management requires regular recording, consistent classification and an understanding of income–expense history. Financial literacy is directly associated with long-term planning and decision-making [1]. Conventional forms, however, require many steps, so omissions or category errors propagate into balances, budgets and reports.

Natural input such as “ăn phở sáng nay 45k bằng Momo” reduces effort but creates new risks. A system must understand Vietnamese currency notation, relative dates, multiple transactions in one sentence, approximate wallet names and classification context. Receipt images and speech introduce additional OCR/STT error. Writing an inferred result directly to the database can therefore corrupt all downstream analysis.

PERFIN follows one principle: PostgreSQL and deterministic functions are the source of truth; LLM, OCR and STT services only produce intermediate data for review. The project addresses three research questions:

1. How can text, image and speech be transformed into structured transactions without compromising data correctness?
2. Which deterministic algorithms are suitable for explainable insights from personal-finance history?
3. Where should an LLM participate to improve natural interaction without taking over computation or modifying data autonomously?

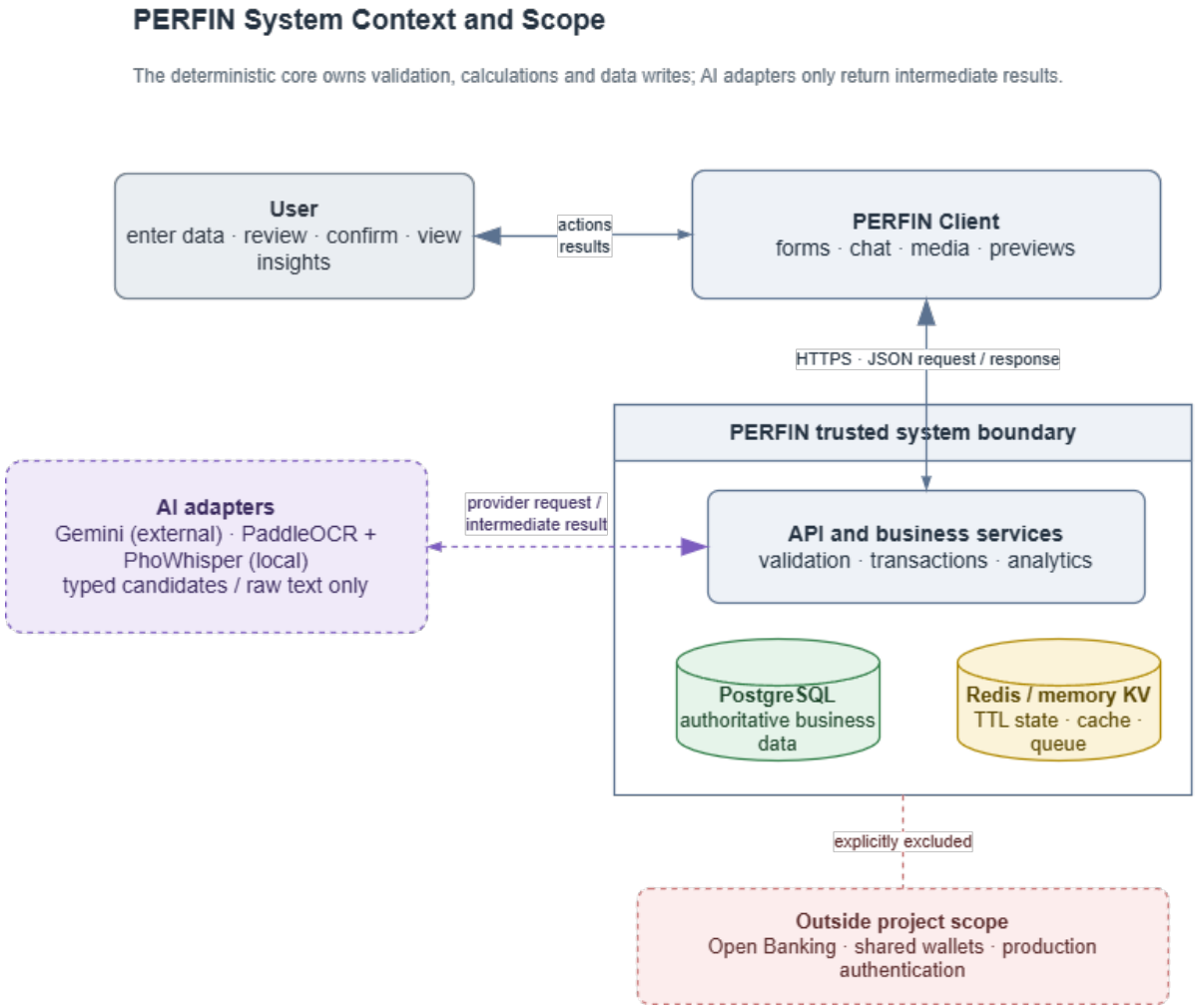


**Figure 1.1:** PERFIN system context and scope; LLM, OCR and STT services remain outside the trusted data boundary.

Figure 1.1 limits the project to acquiring, organizing and analysing personal-finance data. Bank connectivity, shared wallets, production authentication and high-availability infrastructure are out of scope.

## 1.2. OBJECTIVES

The overall objective is to complete, within 13 weeks, a personal-finance prototype in which structured data and deterministic algorithms form the foundation and the LLM is a controlled language-understanding and narration layer.

**Table 1.1:** Specific objectives, acceptance criteria and deadlines

| Code | Specific objective | Evaluation criterion | Deadline |
|---|---|---|---|
| O1 | Build a financial data model with keys, constraints and atomic updates. | Migrations, ERD and rollback tests; no partial write in critical flows. | Week 5 |
| O2 | Build text, image and speech extraction with clarification, preview and confirmation. | No database write when fields are missing; retain source and intermediate data. | Week 8 |
| O3 | Implement trend, anomaly, runway, recurring, correlation, budget and goal algorithms. | Normal, boundary and invariant cases agree with hand-computed results. | Week 9 |
| O4 | Restrict the LLM to intent understanding, tool calling and fact narration. | Tool schema, validation, fallback and confirmation remain separate from write authority. | Week 10 |
| O5 | Evaluate the prototype with a simplified hybrid test strategy. | Use automated tests for the data/algorithm core and manual functional cases for main flows; state mock/live and UAT boundaries. | Week 13 |

The criteria in Table 1.1 are planned acceptance conditions. A result is called "measured" only when its input, environment and artifact are available.

## 1.3. RESEARCH METHODS

The project combines four methods:

1. **Literature review:** synthesize relational data, text similarity, explainable statistics, OCR, STT and function calling to establish the LLM boundary.

2. **Requirements and systems analysis:** study the input–confirmation–analysis process and model actors, data, constraints and ledger-corruption failure modes.
3. **Prototype design:** develop in short iterations; separate route–service–model responsibilities and implement formulas as independently testable deterministic functions.
4. **Testing:** use unit/regression tests over controlled fixtures, internal integration checks with mocks or in-memory substitutes, and manual functional scenarios conducted by the author. Conclusions remain within scope; author testing is not treated as UAT or OCR/STT accuracy.

## 1.4. IMPLEMENTATION PLAN

The plan starts on 11 May 2026 and spans 13 weeks. Testing and report writing may overlap, but each week has a verifiable deliverable.

**Table 1.2:** Thirteen-week implementation plan

| Week | Dates | Main work | Deliverable |
|---|---|---|---|
| 1 | 11–17 May | Review the problem, guideline and related applications. | Research questions and scope. |
| 2 | 18–24 May | Elicit requirements and normalize the SRS. | FR/NFR list and overall use case. |
| 3 | 25–31 May | Design architecture and the LLM boundary. | Architecture diagram and module contracts. |
| 4 | 01–07 Jun | Design the domain and relational schema. | Class diagram, ERD and migration draft. |
| 5 | 08–14 Jun | Implement ledger, constraints and transactions. | O1 and core-data tests. |
| 6 | 15–21 Jun | Implement parser, normalization and fuzzy matching. | Text pipeline and quality gate. |
| 7 | 22–28 Jun | Integrate LLM, clarification and pending state. | Tool schema and preview/confirm flow. |
| 8 | 29 Jun–05 Jul | Integrate OCR/STT through adapters. | Media pipeline and provider-error handling. |
| 9 | 06–12 Jul | Implement analytics, budget and goal planning. | O3 and formula unit tests. |

| Week | Dates | Main work | Deliverable |
|---|---|---|---|
| 10 | 13–19 Jul | Complete narration, feedback and worker logic. | O4, fallback and side-effect controls. |
| 11 | 20–26 Jul | Run automated core tests, define manual functional cases and profile the data. | Test logs, manual checklist and data summary. |
| 12 | 27 Jul–02 Aug | Restructure the report and normalize diagrams. | Guideline-compliant LaTeX v2. |
| 13 | 03–09 Aug | Review, compile, repair references and prepare the demo. | Submission PDF, source and user guide. |

## 1.5. PROJECT SCOPE

The implementation includes income and expense transactions, wallet transfers, categories, budgets, recurring bills, goals, data analysis and prototype-level text/image/speech input. Evaluation focuses on the data model, transformation correctness and algorithms. The mobile interface captures input and demonstrates the workflow.

Open Banking, foundation-model training, shared wallets, investment advice, production authentication and authorization, high availability and uptime commitments are excluded. The prototype uses `default_user`; user foreign keys indicate an extension path, not evidence of secure multi-user operation.

## 1.6. CONTRIBUTIONS

The principal contributions are a constrained data model with atomic updates; a multimodal human-in-the-loop pipeline; independently testable analytics; an LLM boundary limited to extraction and narration; and a hybrid test strategy that separates core correctness from interface-flow checks.

## 1.7. REPORT STRUCTURE

Chapter 1 presents the problem, objectives, methods and plan. Chapter 2 summarizes theory, compares alternatives and explains technology choices. Chapter 3 presents the SRS, design, detailed algorithms and testing. Chapter 4 reviews the objectives, limitations and future work.

# CHAPTER 2: THEORETICAL BACKGROUND

This chapter summarizes the concepts, models and technologies used by the project. Project-specific formulas and thresholds are moved to the detailed design in Chapter 3.

## 2.1. FINANCIAL DATA AND PREPROCESSING

### 2.1.1. Relational Ledger and Data Integrity

A financial transaction minimally contains an amount, type, date, description, category and wallet. Income increases a balance, expense decreases it, and a wallet transfer creates two opposite changes that must not be counted as income or expense. Multi-record changes belong in one database transaction so they either complete or roll back together.

A relational database provides foreign keys, decimal types, domain constraints and aggregate queries. Durable business data must remain separate from short-lived clarification, preview, cache and queue state. Analyses defined on a fixed calendar axis zero-fill inactive periods: trends use completed months, correlations use completed ISO weeks, and runway/anomaly preserve every calendar day. Partial current months/weeks are excluded from comparisons; dropping zero periods distorts slopes, burn rate and correlation.

### 2.1.2. Normalization and Text Similarity

Input is normalized for case, punctuation, whitespace, currency units and relative dates before schema mapping. For strings $a, b$, Levenshtein similarity [2] is

$$s_{edit} = 1 - \frac{D_{lev}(a, b)}{\max(|a|, |b|)}. \tag{2.1}$$

For token sets $T_a, T_b$, the Dice coefficient [3] is

$$s_{dice} = \frac{2|T_a \cap T_b|}{|T_a| + |T_b|}. \tag{2.2}$$

Levenshtein distance captures nearby typing errors, whereas Dice is useful when phrase order or components differ. PERFIN combines exact matches, aliases, both scores and a safety margin between the two best candidates. The chosen thresholds are detailed in Section 3.2.3.2.

## 2.2. EXPLAINABLE ANALYTICAL TECHNIQUES

### 2.2.1. Trend, Anomaly and Correlation

Ordinary least squares (OLS) [4] models a time series as $\hat{y}_i = a + bx_i$. Its slope is

$$b = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}. \tag{2.3}$$

The sign of $b$ gives direction and $R^2$ gives linear fit. OLS is explainable and testable but does not model seasonality or future events. A z-score compares an observation with the mean in standard-deviation units, while the interquartile range is more robust to extremes [5]. PERFIN uses both as review signals, not as fraud conclusions.

Pearson's coefficient [6] measures linear co-movement:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}. \tag{2.4}$$

Correlation is interpreted only within the observed window and does not imply causation. The coefficient is undefined with fewer than three paired observations or when either series has zero variance; that case must be represented as a null value rather than being reported as $r = 0$. PERFIN also removes calendar weeks in which both categories are zero and reports the effective paired support, preventing shared absence from being mistaken for co-movement.

### 2.2.2. Runway, Recurring Items, Budgets and Goals

Runway extrapolates how long liquid balance in the reporting currency can support the recent spending rate; it is not total wealth. Recurring-item discovery groups transactions by description, amount and date interval. Budget forecasting uses spending pace over elapsed days, while goal planning uses the remaining amount, deadline, contribution and debt simulation. These techniques prioritize transparent assumptions and boundary handling. PERFIN's formulas and windows appear in Section 3.2.3.3.

**Table 2.1:** Selected analytical techniques

| Technique | Role | Reason for selection |
|---|---|---|
| OLS | Trend and short-horizon forecast. | Few parameters; slope and fit are explainable. |
| Z-score + IQR | Unusual amounts or days. | Combines distribution sensitivity with quantile robustness. |
| Runway | Cash-depletion warning. | Direct calculation with explicit assumptions. |
| Rule-based grouping | Recurring-spending discovery. | Suits small data and requires no training. |
| Pearson | Co-moving category pairs. | Standardized coefficient with clear interpretation. |
| Budget/Goal planner | Limits, forecasts and progress. | Hand-checkable results and what-if simulation. |

## 2.3. OCR, STT AND LANGUAGE MODELS

### *2.3.1. OCR and Speech-to-Text*

OCR generally includes image preprocessing, text-region detection, recognition and post-processing; PP-OCR is a representative compact OCR pipeline [17]. STT converts audio to a transcript; Whisper and Vietnamese PhoWhisper illustrate large-scale and language-adapted recognition [8, 18]. PERFIN runs PaddleOCR and PhoWhisper locally on the backend host. Both services only return raw text and provider metadata; the result follows the same normalization, preview and confirmation pipeline as manual input. Extracted text may subsequently be sent to Gemini for language parsing only when that provider is configured.

Quality requires ground truth. CER/WER measure text error, while transaction-field accuracy measures the effect on amount, date, category and line items. A smoke test on a few files establishes provider execution, not accuracy.

### *2.3.2. Local Parser and LLM*

Transformers use attention to model token relationships and underpin modern LLMs [9]. Multimodal models such as Gemini extend this ability to text and media [10]. Structured output constrains shape but cannot prove that an amount is correct, a category belongs to the user or an operation is authorized.

**Table 2.2:** Comparison of the local parser and an LLM

| Criterion | Local parser | LLM via function calling |
|---|---|---|
| Language coverage | Strong on known patterns; weak on free-form input. | Better at variations and context. |
| Reproducibility | High, offline and no API cost. | Model/provider dependent and slower. |
| Control | Rules and failures are traceable. | Requires locked schema, validation and fallback. |
| PERFIN role | Baseline, fallback and quality gate. | Primary extraction/routing when configured. |
| Business authority | Returns a typed draft; never writes directly. | Proposes a tool and arguments; never writes directly. |

PERFIN uses provider-schema function calling [11]. The LLM proposes a tool and arguments; the backend validates, builds a preview and waits for confirmation. For insight narration, the LLM only receives precomputed facts. This preserves interaction benefits without making the model a source of truth.

## 2.4. TECHNOLOGIES AND RATIONALE

**Table 2.3:** Technologies, roles and alternatives

| Technology | Reason for selection | Limitation or alternative |
|---|---|---|
| React Native + Expo | One codebase for forms, chat, image, audio and mobile preview. | Separate native apps are only needed for device-specific requirements. |
| Node.js + Express | Fits I/O-oriented APIs and shares JavaScript with business functions and tests. | A modular monolith is sufficient; microservices add operations cost. |
| PostgreSQL [12] | Foreign keys, `DECIMAL`, transactions and aggregates fit a ledger. | File/NoSQL designs provide weaker constraints and rollback. |
| Redis [13] | TTL for pending state, clarification, cache and queue data. | Never the system of record; memory fallback is development-only. |
| BullMQ [14] | Scheduling, retry and job IDs for idempotent work. | Basic cron lacks equivalent queue and retry controls. |

| Technology | Reason for selection | Limitation or alternative |
|---|---|---|
| Gemini + local parser | Combines language coverage with a testable fallback. | Provider adapters prevent business-logic lock-in. |
| PaddleOCR + PhoWhisper | Local execution keeps raw media on the backend host and makes the failure mode explicit. | The Python runtime/model must be installed; unavailable inference returns an error rather than fabricated text. |

## 2.5. RELATED APPLICATIONS AND SELECTED DIRECTION

Money Lover [15] and MISA MoneyKeeper [16] represent form- and dashboard-oriented finance applications. They are strong at structured records but require many entry steps. Chatbots reduce entry work yet risk untraceable numbers; specialist analytics are often difficult for general users.

PERFIN does not attempt commercial feature completeness. It targets the intersection of a relational ledger, deterministic algorithms, human confirmation and a bounded language layer. This direction fits the project's data-and-algorithm focus and allows each part to be evaluated independently.

**Table 2.4:** Research gaps and PERFIN responses

| Gap | Risk | Selected response |
|---|---|---|
| Natural input may be miswritten | Dirty data propagates to reports | Typed draft, validation, preview and confirmation. |
| Media provides raw text only | Wrong amount, date or category | Ground truth, field accuracy and confirmation. |
| Chatbot numbers lack provenance | Hallucination and weak auditability | SQL/analytics creates facts; narrator only explains. |
| Classification does not adapt | Repeated errors | Thresholded, controlled correction retrieval. |
| Retry duplicates effects | Corrupt data or duplicate alerts | Job IDs, fingerprints and idempotent operations. |

# CHAPTER 3: APPLICATION RESULTS

## 3.1. SOFTWARE REQUIREMENTS SPECIFICATION

### *3.1.1. Overall Description*

PERFIN is a mobile prototype for entering, normalizing, managing and analysing personal-finance data. Chat and media complement forms as input channels. The backend performs every financial calculation, constraint check and data change; LLM, OCR and STT services have no direct PostgreSQL access and are not authoritative numeric sources. FR/NFR identifiers and specifications follow the requirements-engineering guidance of ISO/IEC/IEEE 29148:2018 [28].

**Table 3.1:** Product context and constraints

| Item | Description |
|---|---|
| User | A general user records and interprets personal-finance data without requiring statistical expertise. |
| Environment | React Native/Expo on mobile or web, Express API and PostgreSQL; Redis and a worker are configuration-dependent. |
| System of record | PostgreSQL stores business data; Redis stores only TTL-bound pending, clarification, cache and queue state. |
| Constraints | The demo uses `default_user`, has no production authentication/authorization, defaults to VND and has no foreign-exchange conversion. |
| Dependencies | Gemini is an external language provider; PaddleOCR and PhoWhisper run locally on the backend host. Local-media failure returns a real error, never fabricated data. |
| Assumption | The user reviews previews; guidance does not replace professional financial advice. |

**Table 3.2:** Actors and concerns

| Actor | Role | Concern or limitation |
|---|---|---|
| User | Enters, edits, confirms, manages and views reports. | Language/media-derived data must be reviewed before storage. |
| External/local AI adapters | Gemini returns a typed draft or narration; local media runtimes return OCR text or a transcript. | No adapter executes business operations or writes the database; extracted text is still untrusted. |
| Background worker | Runs reminders, analysis and file cleanup. | Jobs require user scope, retry and duplicate-effect protection. |
| Developer administrator | Configures environment, migrations and providers. | Not a business actor of the application. |

Shared rules are: business amounts must be finite and valid; transaction dates cannot be in the future; wallets and categories belong to the current profile; negative wallet balances are allowed; chat/media operations require preview and confirmation; multi-record updates are atomic; reports exclude soft-deleted transactions by default; and insufficient data yields a warning or `null`, never an invented value.

### *3.1.2. Functional Requirements*

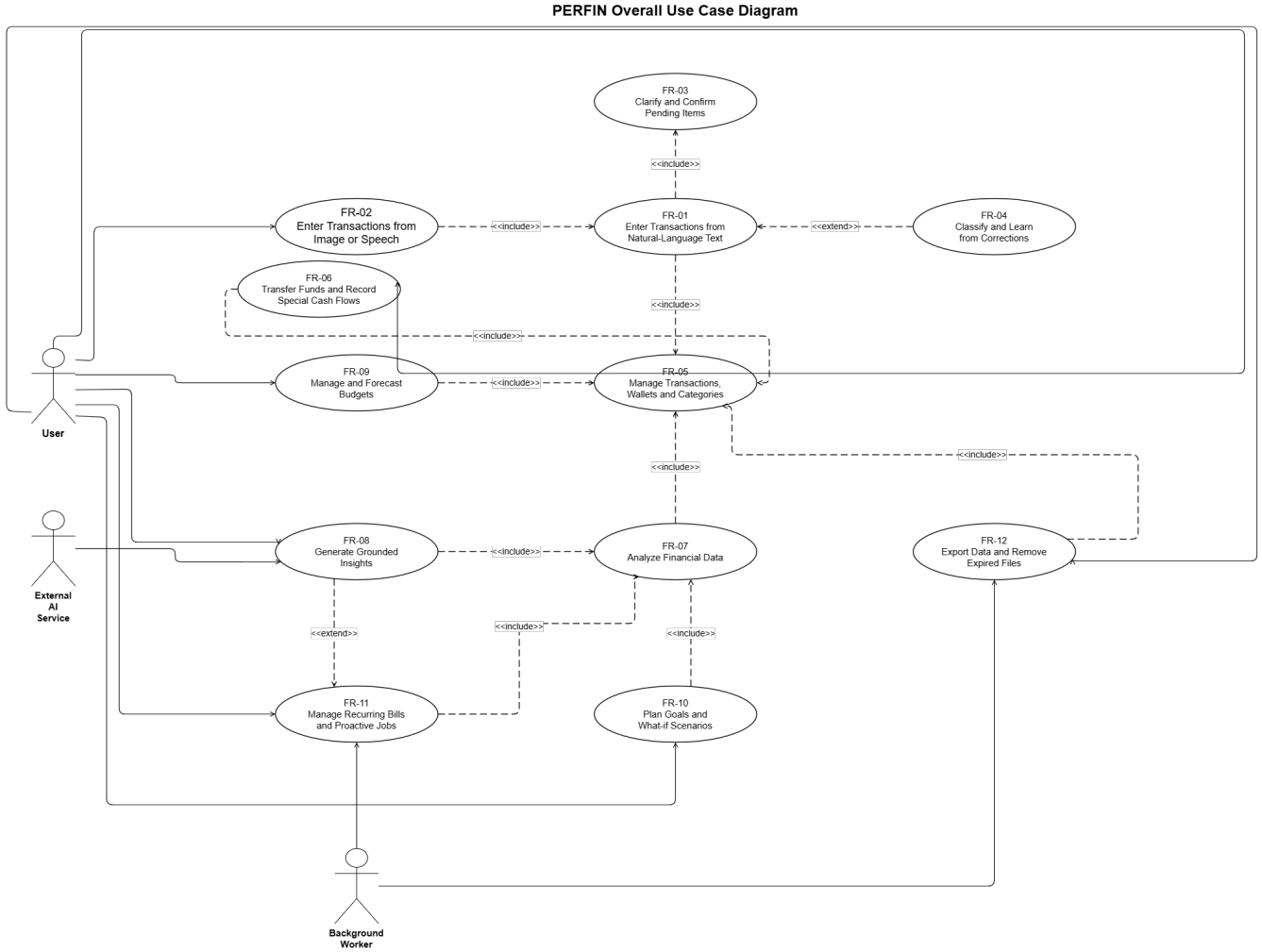


**Figure 3.1:** Overall PERFIN use case diagram. Feature details are specified with flow tables instead of twelve separate diagrams.

Figure 3.1 contains observable actor goals only. Parser, validation, transaction, TTL and retry behaviour belongs in the flow tables or the sequence/flow diagrams in Section 3.2.3.

**Table 3.3:** Functional-requirement catalogue and location of the full specification

| ID | Capability | Priority | Full specification |
|---|---|---|---|
| FR-01 | Natural-language transaction entry | Mandatory | Chapter 3, Table 3.4 |
| FR-02 | Image and speech transaction entry | Desirable | Chapter 3, Table 3.5 |
| FR-03 | Clarification and confirmation | Mandatory | Chapter 3, Table 3.6 |
| FR-04 | Classification and correction retrieval | Mandatory | Chapter 3, Table 3.7 |
| FR-05 | Financial-data management | Mandatory | Chapter 3, Table 3.8 |
| FR-06 | Wallet transfer and special cash flows | Desirable | Appendix, Table B.1 |
| FR-07 | Deterministic financial analytics | Mandatory | Chapter 3, Table 3.9 |
| FR-08 | Grounded insights and narration | Desirable | Chapter 3, Table 3.10 |
| FR-09 | Budget management and forecasting | Mandatory | Chapter 3, Table 3.11 |
| FR-10 | Goal planning and what-if scenarios | Mandatory | Chapter 3, Table 3.12 |
| FR-11 | Recurring bills and proactive jobs | Desirable | Appendix, Table B.2 |
| FR-12 | Export and expired-file cleanup | Optional | Appendix, Table B.3 |

**Table 3.4:** FR-01 — Enter Transactions from Text

| Item | Description |
|---|---|
| **Feature name** | Enter transactions from natural-language text. |
| **ID** | FR-01. |
| **User** | User. |

| Item | Description |
|---|---|
| **Necessity** | Mandatory. |
| **Classification** | Complex. |
| **Participants and concerns** | The user needs fast entry; the AI Orchestrator returns a typed draft; Transaction Service protects the ledger. |
| **Summary** | Convert one sentence into one or more structured transactions and create a preview before writing. |
| **Relationships** | Includes FR-03 and FR-04; uses FR-05 after confirmation. |
| **Normal flow** | **Step 1:** User submits text.<br>**Step 2:** Extract and normalize.<br>**Step 3:** Validate schema, category and wallet.<br>**Sub 1:** Build preview.<br>**Step 4:** User confirms.<br>**Step 5:** Commit atomically and return the result. |
| **Subflows** | **Sub 1 — Preview:** Store a TTL-bound draft.<br>**Sub 1.1:** Show amount, type, date, category, wallet and warnings. |
| **Alternate flows** | **Step 2:** Provider failure uses the local parser or returns an error.<br>**Step 3:** Missing/ambiguous fields invoke FR-03.<br>**Step 4:** Edit or cancel causes no write. |

**Table 3.5:** FR-02 — Enter Transactions from Image or Speech

| Item | Description |
|---|---|
| **Feature name** | Enter a transaction from a receipt image or speech. |
| **ID** | FR-02. |
| **User** | User; local media runtime; optional external language adapter. |
| **Necessity** | Desirable. |
| **Classification** | Complex. |
| **Participants and concerns** | The user needs visible raw text; local OCR/STT reports provider and failure; the backend rejects invalid files. |
| **Summary** | Convert image/audio into reviewable text, then reuse FR-01. |
| **Relationships** | Includes FR-01 and FR-03. |

| Item | Description |
|---|---|
| **Normal flow** | **Step 1:** Select a valid file.<br>**Step 2:** Backend invokes OCR or STT.<br>**Step 3:** Display raw text and provider.<br>**Sub 1:** User edits/confirms text.<br>**Step 4:** Continue with FR-01. |
| **Subflows** | **Sub 1 — Media review:** Edit a voice transcript.<br>**Sub 1.1:** For a receipt, select the total or individual items when present. |
| **Alternate flows** | **Step 1:** An invalid type or size above 10 MB is rejected.<br>**Step 2:** A provider that returns no text produces an error and no mock or pending draft. |

**Table 3.6:** FR-03 — Clarify and Confirm Pending Transactions

| Item | Description |
|---|---|
| **Feature name** | Clarification, preview and pending-transaction confirmation. |
| **ID** | FR-03. |
| **User** | User. |
| **Necessity** | Mandatory. |
| **Classification** | Complex. |
| **Participants and concerns** | The user can edit/cancel; the KV store applies TTL; a preview can be consumed once. |
| **Summary** | Collect missing fields across turns and prevent inferred data from being written. |
| **Relationships** | Used by FR-01, FR-02, FR-09, FR-10 and FR-12. |
| **Normal flow** | **Step 1:** Store intent and missing fields.<br>**Step 2:** Ask for the next field.<br>**Step 3:** Merge and revalidate.<br>**Sub 1:** Build or edit a preview.<br>**Step 4:** Atomically claim pending state.<br>**Step 5:** Call the business service. |

| Item | Description |
|---|---|
| **Subflows** | **Sub 1 — Preview:** Attach a `pending_id` with a five-minute TTL.<br>**Sub 1.1:** Update only with a valid payload. |
| **Alternate flows** | **Step 3:** Continue asking while fields are missing.<br>**Step 4:** Expired, incorrect or claimed IDs cannot write.<br>**Cancel:** Clear state and perform no write. |

**Table 3.7:** FR-04 — Classify and Learn from Corrections

| Item | Description |
|---|---|
| **Feature name** | Category classification and correction retrieval. |
| **ID** | FR-04. |
| **User** | User. |
| **Necessity** | Mandatory. |
| **Classification** | Complex. |
| **Participants and concerns** | The matcher must be explainable; the user corrects labels; feedback failure must not corrupt a committed transaction. |
| **Summary** | Select a category by exact/alias/fuzzy/LLM methods and reuse confirmed corrections. |
| **Relationships** | Extends FR-01 and may be triggered by an FR-05 category edit. |
| **Normal flow** | **Step 1:** Normalize the description.<br>**Step 2:** Load same-profile categories and corrections.<br>**Step 3:** Rank candidates.<br>**Sub 1:** Apply a strong correction.<br>**Step 4:** Return category, confidence and match kind.<br>**Step 5:** Store feedback after confirmation. |
| **Subflows** | **Sub 1 — Correction:** Select at most five similar corrections.<br>**Sub 1.1:** Use them as context or a controlled override. |
| **Alternate flows** | **Step 3:** A low score, small margin or conflicting corrections return Other or request a selection.<br>**Feedback write:** A failure is logged only. |

**Table 3.8:** FR-05 — Manage Financial Data

| Item | Description |
|---|---|
| **Feature name** | Manage transactions, wallets and categories. |
| **ID** | FR-05. |
| **User** | User. |
| **Necessity** | Mandatory. |
| **Classification** | Complex. |
| **Participants and concerns** | The user needs CRUD and restore; services maintain balance and history consistency. |
| **Summary** | Create, view, edit, soft-delete and restore transactions and manage profile-scoped categories/wallets. |
| **Relationships** | Used by FR-01/FR-02 and supplies FR-07–FR-12. |
| **Normal flow** | **Step 1:** Select an operation.<br>**Step 2:** Validate payload and ownership.<br>**Step 3:** Compute the balance delta.<br>**Sub 1:** Update the transaction and wallet atomically.<br>**Step 4:** Return the record and new balance. |
| **Subflows** | **Sub 1 — Update:** Write the transaction.<br>**Sub 1.1:** Update the wallet.<br>**Sub 1.2:** Commit, then invalidate cache best-effort. |
| **Alternate flows** | **Step 2:** Out-of-scope IDs or invalid payloads are rejected.<br>**Step 3:** A negative balance remains valid.<br>**Before commit:** Any failure rolls back. |

**Table 3.9:** FR-07 — Analyse Financial Data

| Item | Description |
|---|---|
| **Feature name** | Trend, anomaly, runway, recurring and correlation analysis. |
| **ID** | FR-07. |
| **User** | User; Background worker. |
| **Necessity** | Mandatory. |
| **Classification** | Complex. |
| **Participants and concerns** | Analytics needs clean data; users need period, sample size and method. |

| Item | Description |
|---|---|
| **Summary** | Compute deterministic facts over non-deleted data; trend/correlation use completed periods and fixed calendar axes are zero-filled. |
| **Relationships** | Reads FR-05 and supplies FR-08 and FR-11. |
| **Normal flow** | **Step 1:** Select the analysis and period.<br>**Step 2:** Query profile data.<br>**Step 3:** Normalize the calendar axis and check sample count.<br>**Sub 1:** Run the algorithm.<br>**Step 4:** Return facts with window, method, parameter and warning metadata. |
| **Subflows** | **Sub 1 — Analytics:** Run OLS, z-score/IQR, runway, recurring or Pearson through one result contract. |
| **Alternate flows** | **Sparse data or zero variance:** Return a degraded result or `null`.<br>**Limit:** Make no causal claim and fabricate no transaction. |

**Table 3.10:** FR-08 — Generate Grounded Insights

| Item | Description |
|---|---|
| **Feature name** | Grounded insight and persona narration. |
| **ID** | FR-08. |
| **User** | User; External AI service. |
| **Necessity** | Desirable. |
| **Classification** | Complex. |
| **Participants and concerns** | The user needs clear explanation; Analytics owns facts; the narrator cannot alter numbers. |
| **Summary** | Explain precomputed facts and return them with the narration for traceability. |
| **Relationships** | Includes FR-07 and may use a persona when consent is enabled. |

| Item | Description |
|---|---|
| **Normal flow** | **Step 1:** Receive a question or report request.<br>**Step 2:** Generate facts.<br>**Step 3:** Load a permitted persona.<br>**Sub 1:** Narrate the facts.<br>**Step 4:** Return narration, facts and provider metadata. |
| **Subflows** | **Sub 1 — Narration:** Lock numbers and units in the prompt.<br>**Sub 1.1:** Use a deterministic template when the provider fails. |
| **Alternate flows** | **Sparse facts:** Return a warning.<br>**Limit:** A complete numeric-grounding checker remains a design target and is not presented as measured. |

**Table 3.11:** FR-09 — Manage and Forecast Budgets

| Item | Description |
|---|---|
| **Feature name** | Monthly budgets, recommendations and forecasts. |
| **ID** | FR-09. |
| **User** | User. |
| **Necessity** | Mandatory. |
| **Classification** | Medium. |
| **Participants and concerns** | Users need progress visibility; Budget Engine needs sufficient history and must not apply limits automatically. |
| **Summary** | Compute spent/remaining, forecast month end and propose limits from history or a ratio framework. |
| **Relationships** | Reads FR-05 and may invoke FR-03 from chat. |
| **Normal flow** | **Step 1:** Select a month.<br>**Step 2:** Query spending.<br>**Step 3:** Compute progress and forecast.<br>**Sub 1:** Produce a warning or recommendation.<br>**Step 4:** Confirm before saving a recommendation. |

| Item | Description |
|---|---|
| **Subflows** | **Sub 1 — Recommendation:** Zero-fill months.<br>**Sub 1.1:** Compute limits.<br>**Sub 1.2:** Proportionally shrink raw allocations and reconcile rounding when a group exceeds its cap. |
| **Alternate flows** | **Fewer than three history months:** Produce a warning.<br>**Invalid data:** A non-positive amount or non-expense category is rejected. |

**Table 3.12:** FR-10 — Plan Goals and What-if Scenarios

| Item | Description |
|---|---|
| **Feature name** | Savings, purchase and debt-payoff goal planning. |
| **ID** | FR-10. |
| **User** | User. |
| **Necessity** | Mandatory. |
| **Classification** | Complex. |
| **Participants and concerns** | The user needs formula-backed progress; Goal Planner must expose infeasible assumptions. |
| **Summary** | Calculate contribution, duration, amortization and what-if plans without modifying the original until confirmation. |
| **Relationships** | Reads FR-05/FR-07 and may use FR-03. |
| **Normal flow** | **Step 1:** Enter the goal.<br>**Step 2:** Validate amount, date and rate.<br>**Step 3:** Compute allocatable cash.<br>**Sub 1:** Run the goal-specific formula.<br>**Step 4:** Return the plan and warnings.<br>**Step 5:** Save after confirmation. |
| **Subflows** | **Sub 1 — Calculation:** What-if reruns the saving/purchase function with an additional contribution.<br>**Sub 1.1:** Debt payoff uses monthly simulation.<br>**Sub 1.2:** The original plan remains unchanged and no scenario is persisted. |

| Item | Description |
|---|---|
| **Alternate flows** | **Invalid inputs:** Request correction.<br>**Infeasible payment or horizon:** Payment less than or equal to first-month interest warns of negative amortization; zero payment and a 600-month horizon are reported explicitly.<br>**Cancel:** Perform no write. |

### 3.1.3. Non-functional Requirements

**Table 3.13:** Non-functional requirements and acceptance methods

| ID | Group | Testable requirement | Measurement |
|---|---|---|---|
| NFR-01 | Recovery | Provider/Redis failure fabricates no data; restore demo service and backup data within three hours after an incident. | Fault injection, restart and restore on a test database. |
| NFR-02 | Integrity | No partial create, update, delete, restore, transfer or payment; credentials are never transmitted or logged in clear text. | Rollback tests and configuration/log scanning; TLS for network deployment. |
| NFR-03 | UI | Vietnamese blue–slate interface on neutral surfaces, consistent navigation and no overflow at mobile viewports. | Manually inspect the main screens at a mobile viewport for navigation, content and layout overflow. |
| NFR-04 | Timing | Acknowledge within three seconds; target text p95 $\leq 3$ s and media p95 $\leq 8$ s in a disclosed environment. | At least 30 runs per flow with provider, hardware, cache and error rate. |
| NFR-05 | Correctness | Algorithms match expected normal/boundary cases; negative balances are valid; narrator invents no number. | Unit tests, invariants and facts–response checker. |
| NFR-06 | Privacy | Remove temporary media; use traits only with consent; ID queries include user scope. | Temporary-file, consent and cross-scope tests. |

| ID | Group | Testable requirement | Measurement |
|---|---|---|---|
| NFR-07 | Worker consistency | Retrying an event/job creates no duplicate payment, message or export. | Repeat fingerprints and count effects. |
| NFR-08 | Traceability | Analysis includes schema version, timestamp, component value and metadata for unit, window, sample count, method, parameters, warnings and degradation status. | Unit/contract tests and inspection of returned metadata. |

Recovery, latency, OCR/STT accuracy and complete numeric grounding remain acceptance targets; unmeasured targets are not presented as achieved results.

## 3.2. SOFTWARE DESIGN

### *3.2.1. Architecture*

PERFIN is a modular monolith: domain services are separated by responsibility while the API remains one process; a separate worker handles background jobs. This matches project scale, reduces operating cost and preserves independent module tests.

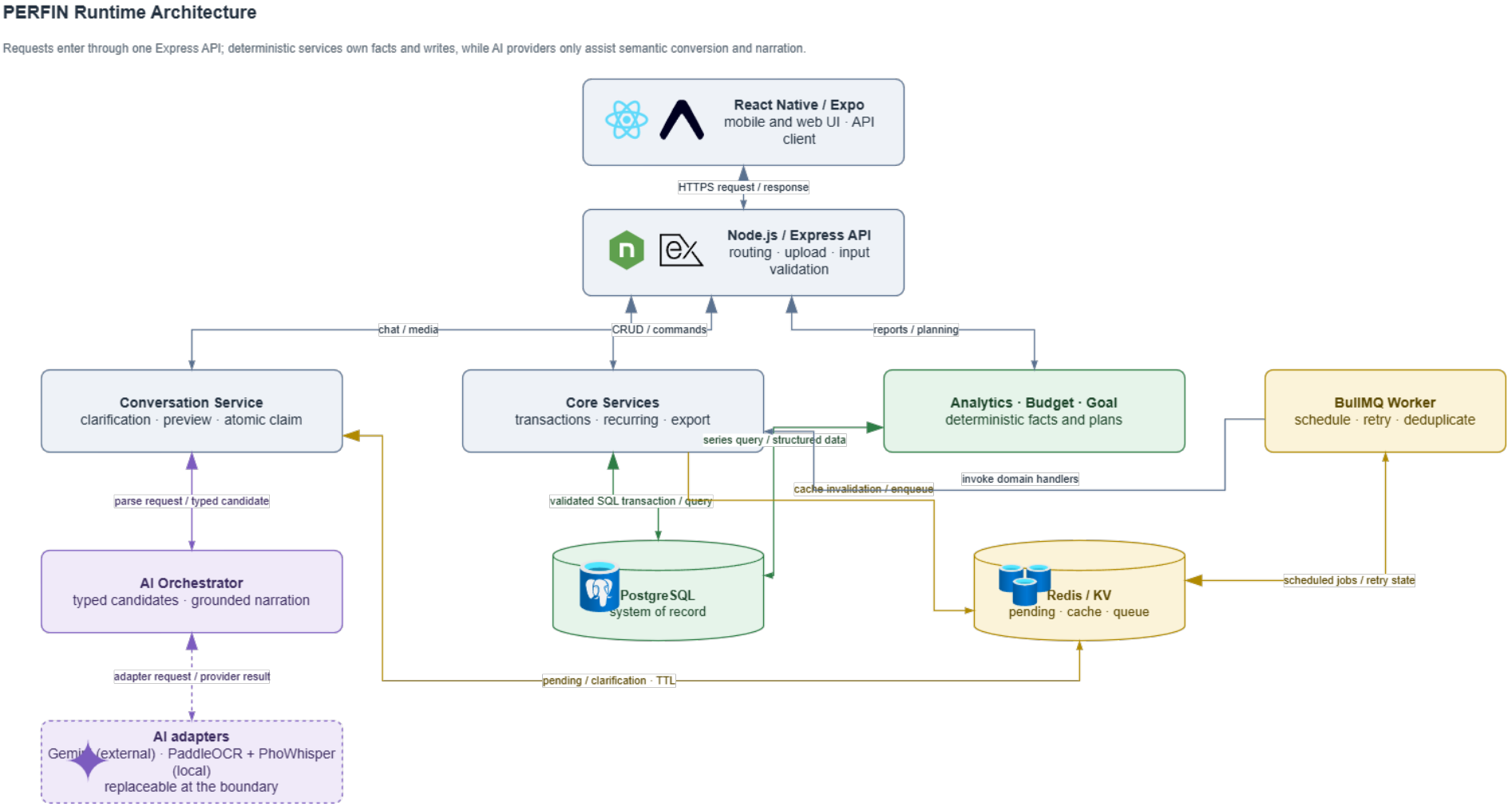


**Figure 3.2:** PERFIN runtime architecture; the deterministic core creates and stores facts while the AI Orchestrator coordinates semantics.

**Table 3.14:** Architectural components and responsibilities

| Component | Responsible for | Not responsible for |
|---|---|---|
| Mobile App | Capture input, show previews/facts and confirm. | Database access or AI secrets. |
| Express Routes | HTTP contracts, upload and input validation. | Analytical formulas. |
| Core Services | Business rules, transactions and idempotency. | LLM-generated numbers. |
| Analytics, Budget, Goal | Deterministic facts and plans. | Arbitrary advice. |
| AI Orchestrator | Tool routing, extraction, narration and fallback. | Direct database writes. |
| PostgreSQL | System of record, constraints and aggregates. | Temporary conversation state. |
| Redis/BullMQ | TTL state, cache, queue and retry. | Financial source of truth. |

Microservices were considered but not selected because prototype scale does not justify deployment, tracing and distributed-consistency costs. Serverless is also a poor fit for a long-running worker and local media providers. A modular monolith keeps database transactions simple and leaves a future extraction path when real load evidence exists.

### *3.2.2. Data Design*

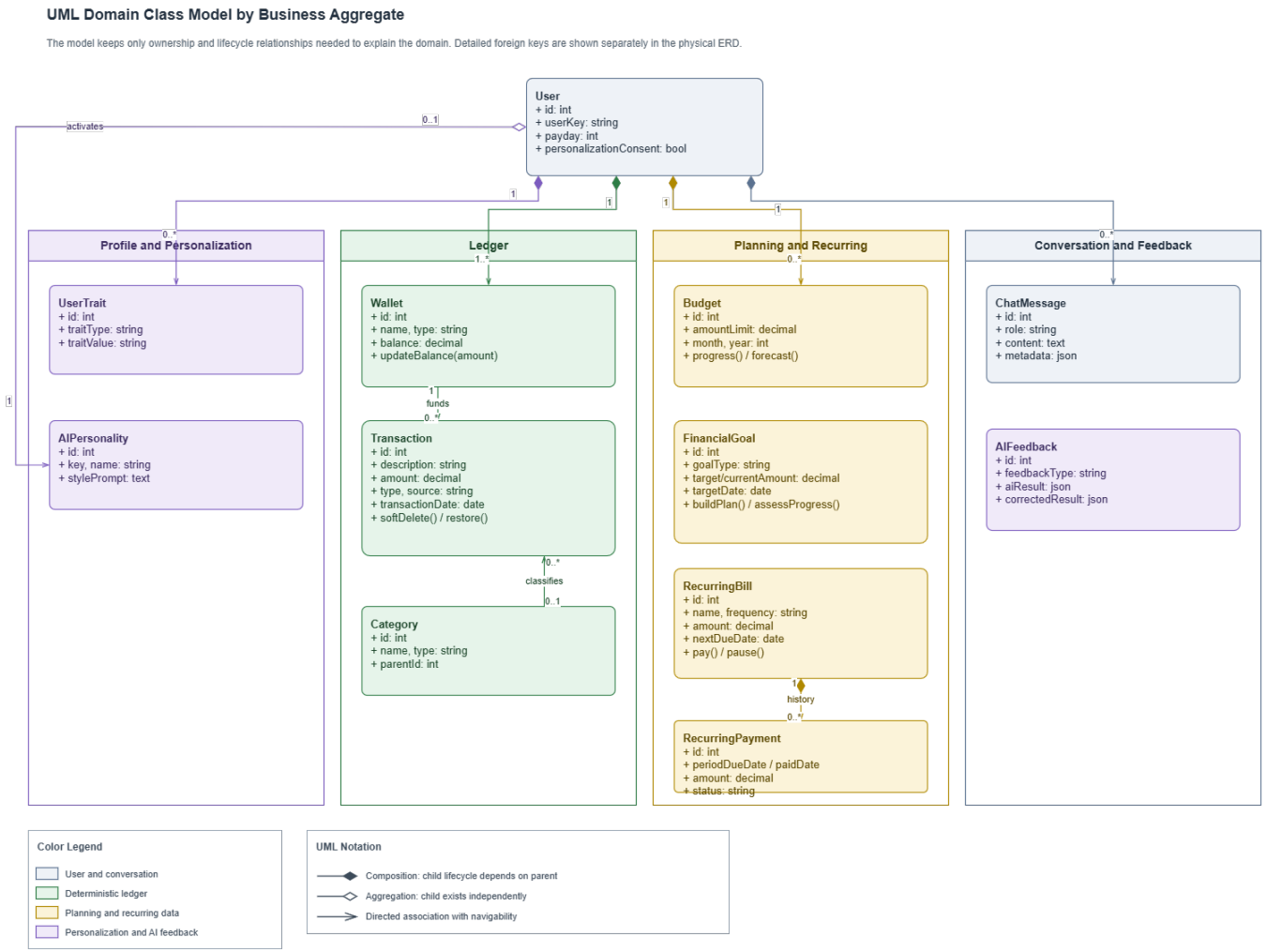


**Figure 3.3:** PERFIN UML domain class model separating ledger, planning, recurring and conversation/feedback aggregates.

`User` is the ownership root; `Wallet`, `Transaction` and `Category` form the ledger; `Budget`, `FinancialGoal` and recurring classes form planning; `ChatMessage` and `AIFeedback` preserve traceable interaction. Engines are not entities because they only read data and create facts.

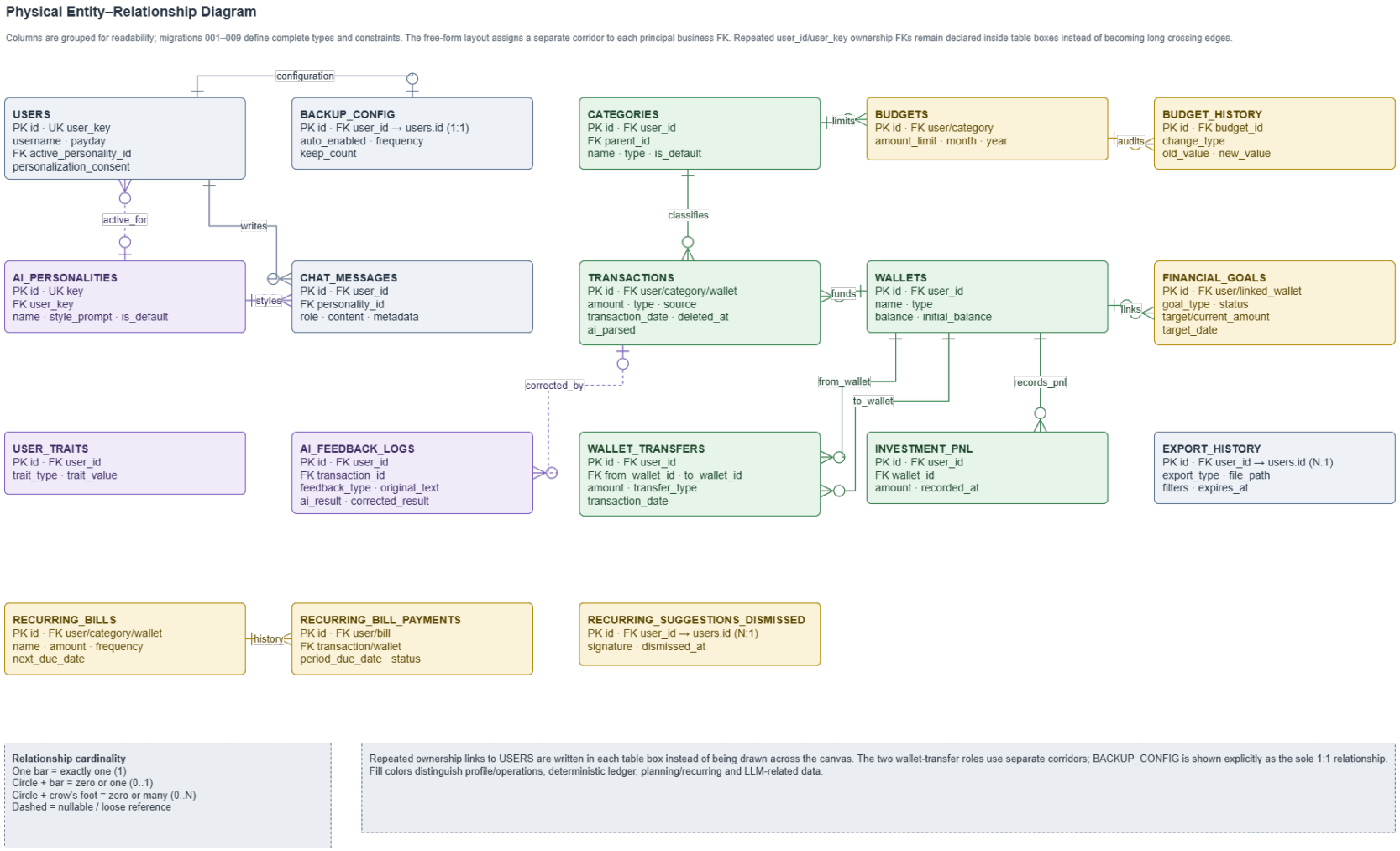


**Figure 3.4:** Physical ERD reconciled with runtime migrations; principal business foreign keys use separate orthogonal corridors.

**Table 3.15:** Entities in alphabetical order

| Entity | Type | Description |
|---|---|---|
| ai_feedback_logs | History | Original AI result, correction and context. |
| ai_personalities | Catalogue | Selectable persona and style prompt. |
| backup_config | Configuration | Per-user backup policy; worker backup is not proven. |
| budget_history | History | Audit trail of budget-limit changes. |
| budgets | Planning | Category and monthly limits. |
| categories | Catalogue | Income/expense taxonomy and parent relation. |
| chat_messages | History | Conversation, facts/metadata and event key. |
| export_history | Operations | Exported files, filters and expiry. |
| financial_goals | Planning | Goal, progress, deadline and linked wallet. |
| investment_pnl | Ledger | Realized investment gain/loss. |
| recurring_bill_payments | Ledger | Payment record for each recurring period. |
| recurring_bills | Planning | Amount, frequency and next due date. |
| recurring_suggestions_dismissed | History | Signature of a dismissed recurring suggestion. |
| transactions | Ledger | Income/expense, input source and soft-delete status. |
| user_traits | Personalization | Consent-dependent user traits. |
| users | Data root | Demo profile and data-scope key. |
| wallet_transfers | Ledger | Transfer between two wallets. |
| wallets | Ledger | Wallet balance, type and name. |

Money uses DECIMAL(15,2); report queries exclude soft-deleted records; deleting a referenced category is blocked or controlled by re-tagging; pending state exists only in Redis; and target-design ID queries include user_id/user_key. Internal transfer requires matching wallet currencies. Runway sums only liquid VND cash,

`bank` and `e_wallet` balances; foreign-currency, savings, investment and credit-card wallets are excluded because the prototype has neither exchange-rate nor liquidity modelling.

### 3.2.3. Detailed Design

#### 3.2.3.1. LLM Boundary and Input

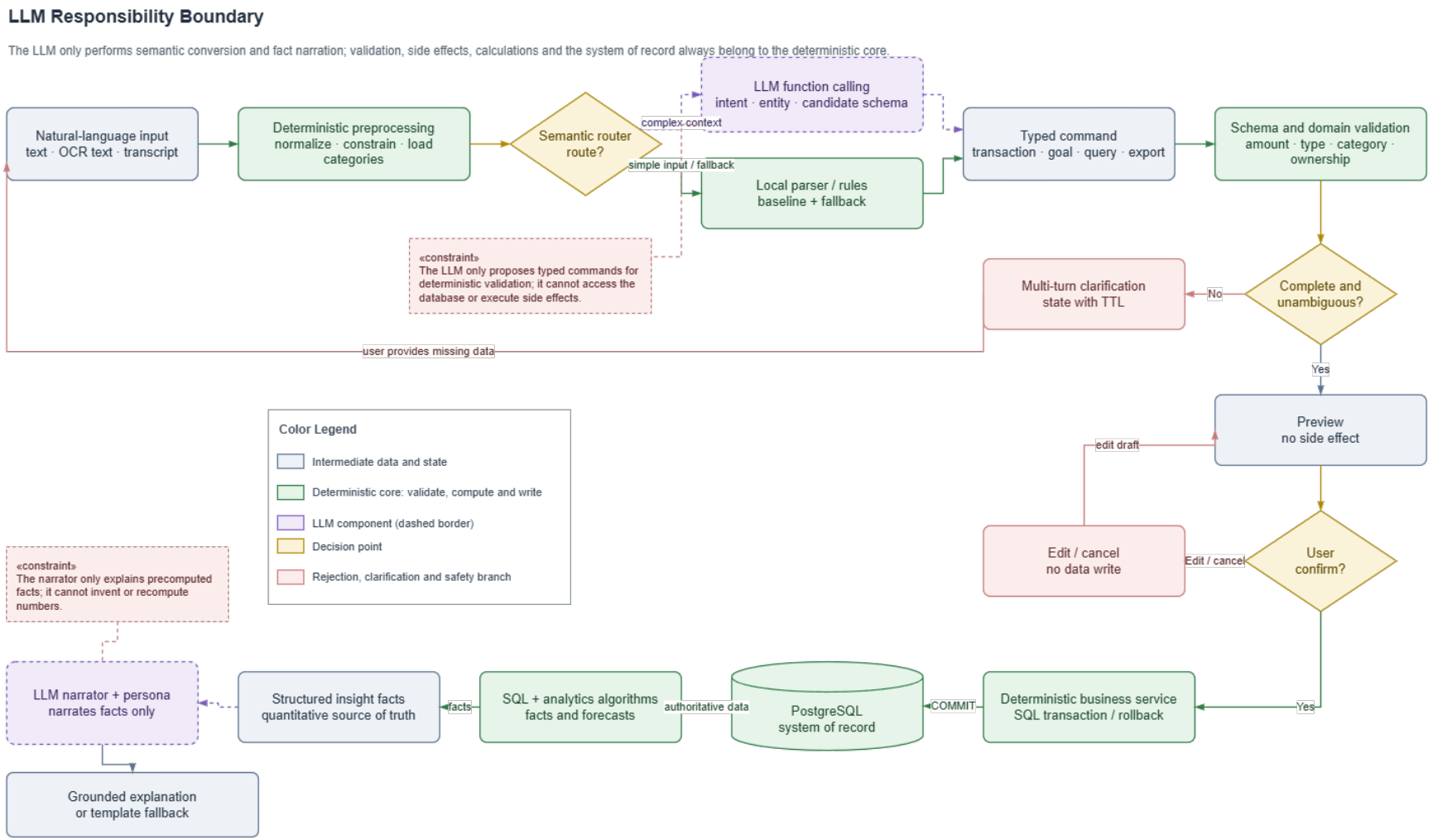


**Figure 3.5:** LLM responsibility boundary: typed drafts and narration remain outside validation, calculation and data-write authority.

The LLM only creates a typed command or narrates facts. The backend validates schema and domain rules, builds a preview and waits for confirmation before opening a transaction. For analysis, SQL and the engine create facts before narration. A parser or deterministic template provides a testable fallback.

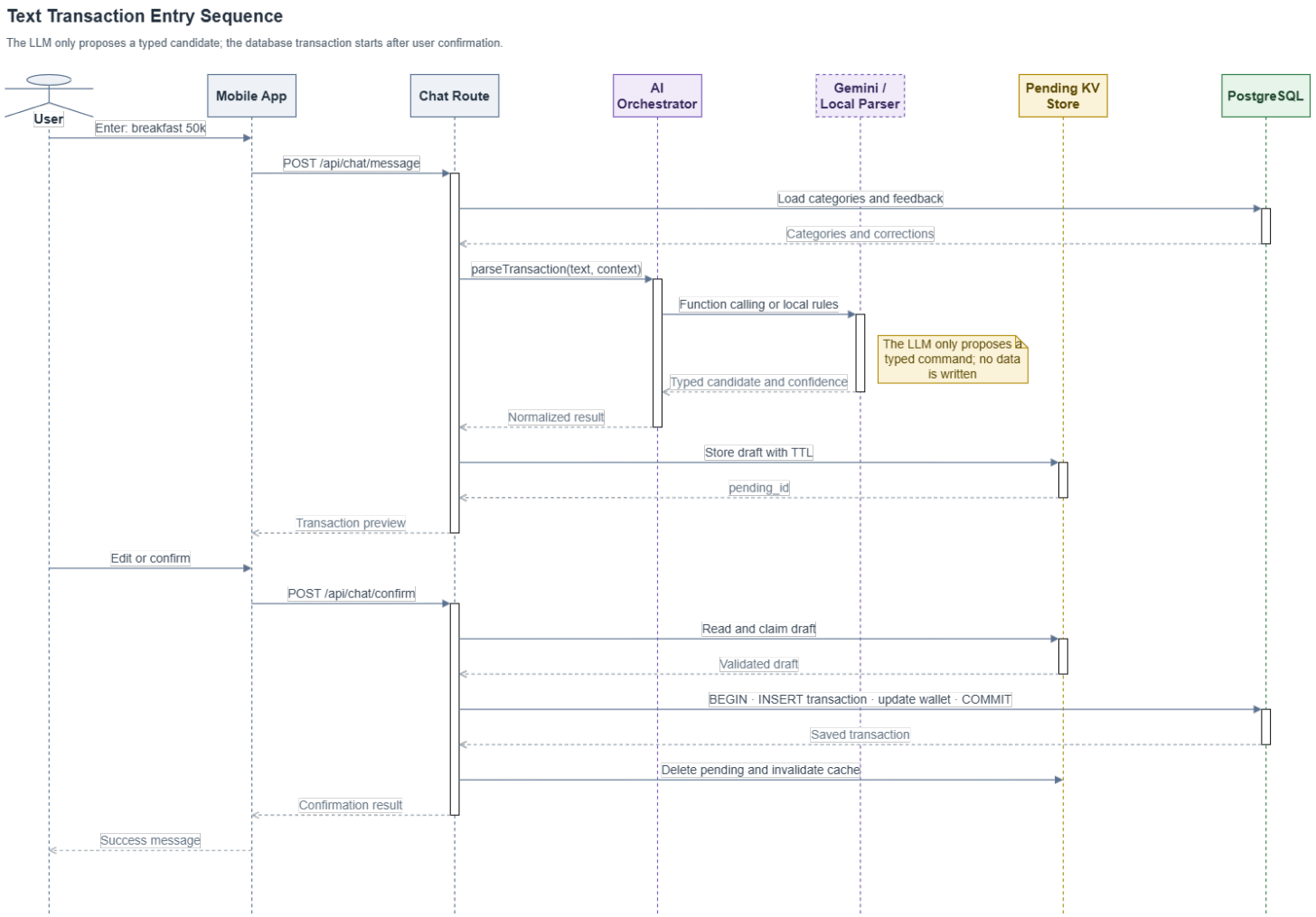


**Figure 3.6:** Text-entry sequence; the data-write transaction starts only after confirmation.

Parsing/preview has no side effect; confirm/commit atomically claims pending state. State contains intent, missing fields, candidates and a draft with a five-minute TTL. Concurrent confirmation must not create two transactions from one `pending_id`.

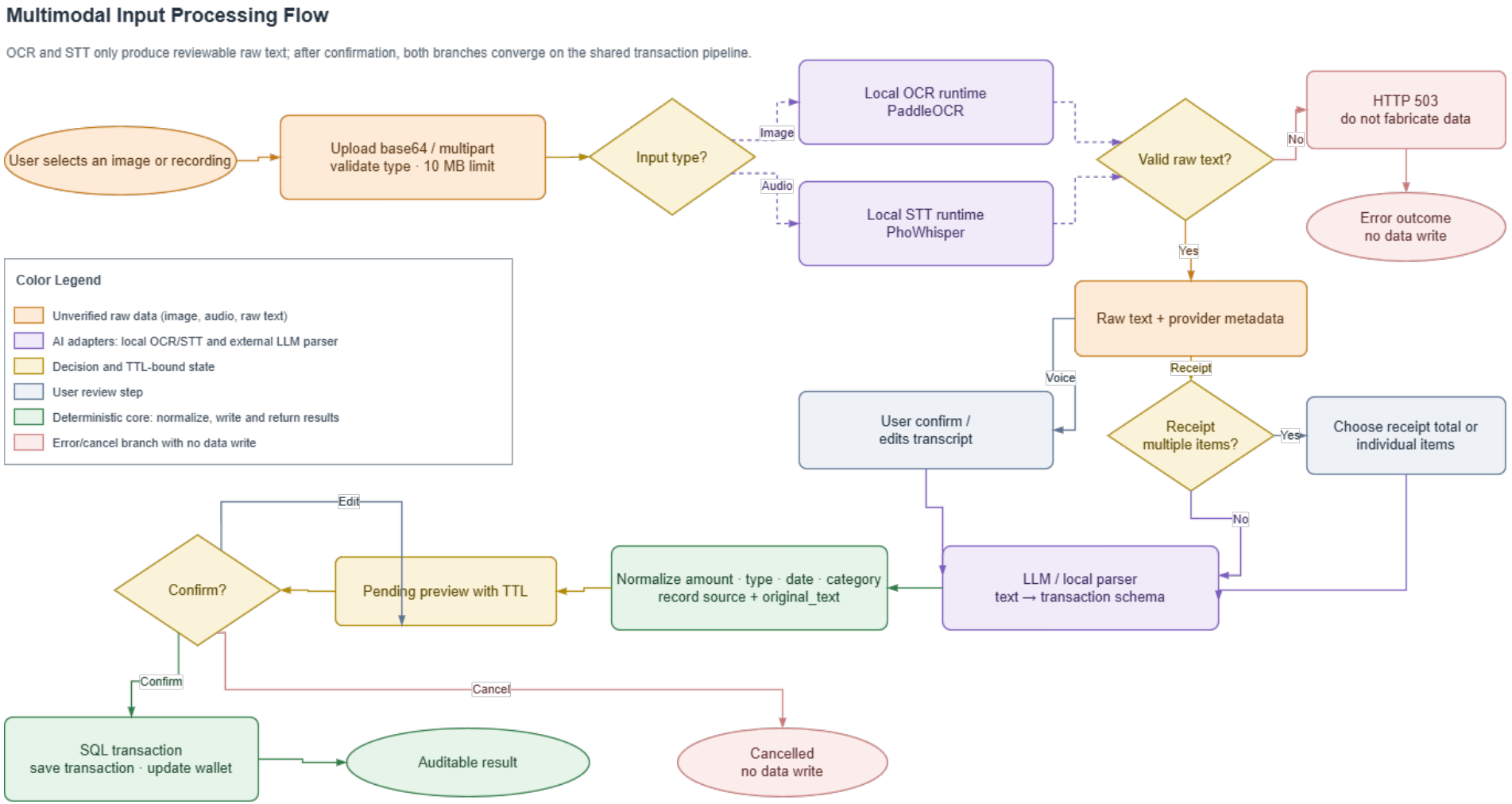


**Figure 3.7:** Image and speech processing; both branches converge on the text pipeline after raw-text review.

OCR/STT returns only raw text and provider metadata. Current ground truth is too small and receipt line totals are not automatically reconciled, so Figure 3.7 is a design flow, not evidence of accuracy.

#### *3.2.3.2. Classification and Correction Retrieval*

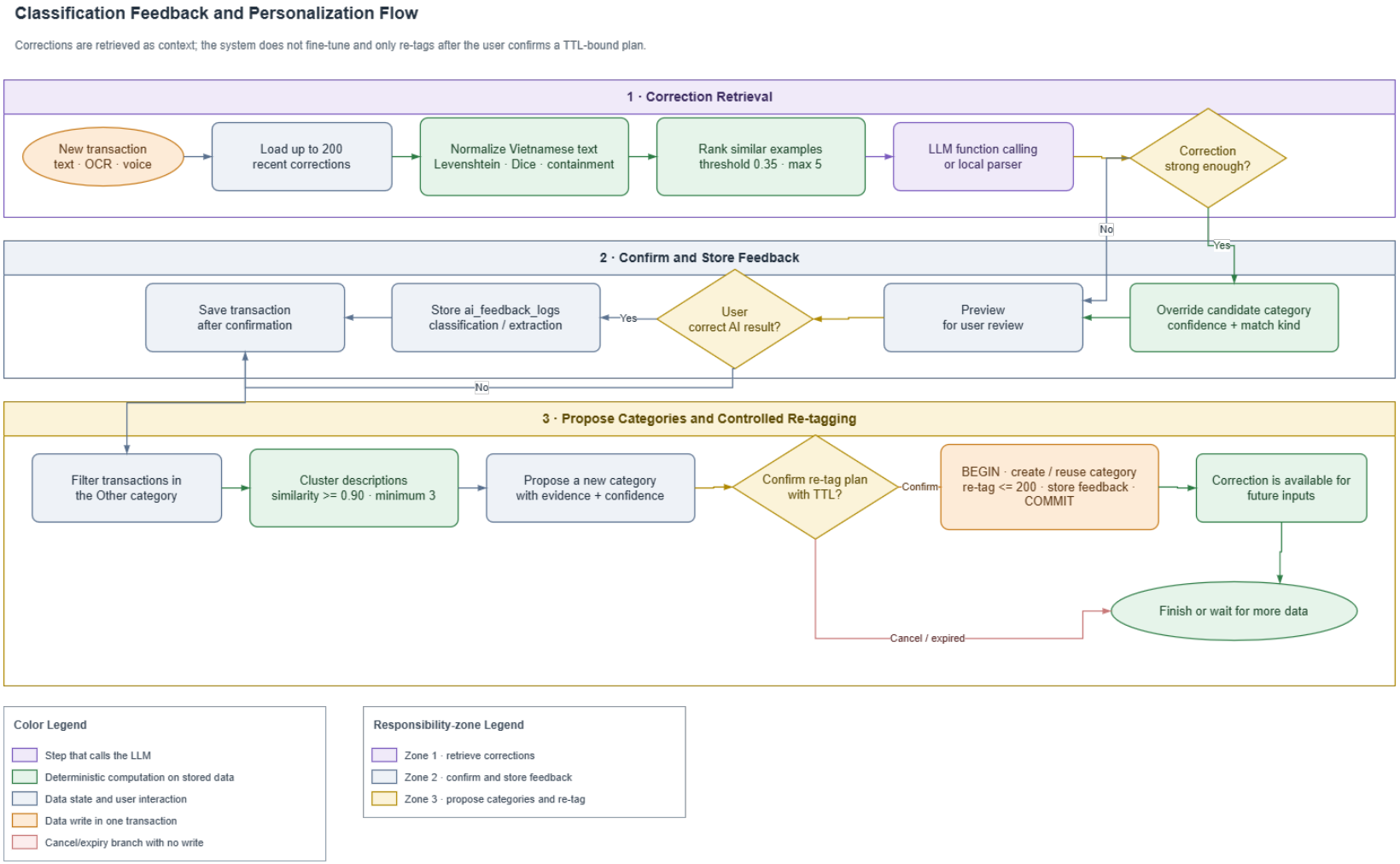


**Figure 3.8:** Classification and correction-retrieval flow; feedback is stored only after confirmation.

The matcher uses

$$s = \max(s_{edit},\ 0.92 s_{dice},\ s_{contain}). \tag{3.1}$$

Let $U, V$ be normalized token sets and choose $|U| \le |V|$. The implementation defines containment as

$$s_{contain} = \begin{cases} \min\left(0.94,\ 0.82 + 0.12\dfrac{|U|}{|V|}\right), & U \subseteq V \text{ and } |U| \ge 2, \\ 0, & \text{otherwise.} \end{cases} \tag{3.2}$$

Selection proceeds through exact name, exact alias and fuzzy matching. Input length is measured after normalization: input longer than four characters uses $\tau = 0.82$, shorter input uses $\tau = 0.90$, and the best candidate must exceed the second by $m = 0.08$. These are prototype starting values, not universal optima. Retrieval loads at most 200 recent corrections and overrides only with a record of the same income/expense type; legacy records without type are ineligible. A fuzzy correction needs at least 0.88 similarity, and competing labels require at least 0.8 agreement for the winner. Multiple labels for one exact text return fallback/clarification instead of majority voting.

#### 3.2.3.3. Analytics and Planning Algorithms

Default windows are 14 calendar days for runway, 30 calendar days for anomaly, 200 days for recurring discovery, six completed months for trend, 60 calendar days for day-of-week analysis and 12 completed ISO weeks for correlation. Fixed day/month/week axes zero-fill inactive periods; an incomplete current month/week is excluded from comparisons. A payday is a local day-of-month and day 31 is clamped to the target month's last calendar day.

Runway uses only liquid VND balance:

$$B = \sum_{\substack{w.currency=\text{VND} \\ w.type \in \{\text{cash,bank,e\_wallet}\}}} balance_w. \tag{3.3}$$

For daily expense $e_j$ and $W = 14$,

$$\bar{e}_W = \frac{1}{W}\sum_{j=1}^{W} e_j, \qquad d = \left\lfloor \frac{B}{\bar{e}_W} \right\rfloor. \tag{3.4}$$

If $B \le 0$, then $d = 0$; if $\bar{e}_W = 0$, the depletion date is undefined. For monthly spending $y_j$, the trend detector keeps the zero-filled six-month axis but requires at least three positive observed months, then uses OLS and emits a signal only when slope

$b > 0$, $R^2 \geq 0.5$, and mean stepwise percentage change (over steps with a positive preceding value) is at least 10%.

Anomaly detection uses the sample standard deviation over all 30 days. It is evaluated only when at least four calendar days have positive spending, and flags only the upper tail when

$$z_i = \frac{e_i - \bar{e}}{s} \geq 2.5 \quad \text{or} \quad e_i > Q_3 + 1.5\,IQR. \tag{3.5}$$

When $s = 0$, the implementation sets $z_i = 0$; when $IQR = 0$, the IQR branch is disabled. Thus a constant series cannot become anomalous merely because the denominator is undefined.

Recurring grouping normalizes descriptions and simultaneously requires at least three occurrences, every $\delta_i = |a_i - \bar{a}|/\bar{a} \leq 0.15$, and gap coefficient of variation $CV_g = \sigma_g/\bar{g} \leq 0.12$ (population $\sigma_g$ over positive gaps). Every gap must fit one cadence: weekly 5–9 days, monthly 26–35 days or quarterly 80–100 days. A candidate is active only when its latest occurrence is no older than that cadence's maximum gap; the output also records `lastSeen` and the calendar-clamped `nextExpected`. There is no default amount ceiling. The monthly estimate is

$$\widehat{a}_{month} = \begin{cases} \bar{a}\,52/12, & \text{weekly}, \\ \bar{a}, & \text{monthly}, \\ \bar{a}/3, & \text{quarterly}. \end{cases} \tag{3.6}$$

Correlation zero-fills a shared 12-week axis, keeps categories observed in at least four weeks and skips undefined Pearson pairs (fewer than three paired observations or zero variance). It returns only the strongest positive pair when $r \geq 0.6$; association does not imply causation. Day-of-week analysis uses the 60-day window, requires at least four observed weekdays and emits a signal only when the peak-to-mean ratio reaches 1.8.

For category spending $s_{c,j}$ across $m$ months, a five-percent-buffer baseline is

$$b_c = 1.05 \left( \frac{1}{m} \sum_{j=1}^{m} s_{c,j} \right). \tag{3.7}$$

Month-end forecast is $\widehat{S}_D = (S/d_e)D$, where $S$ is spending so far, $d_e$ elapsed calendar days including today and $D$ month length. The 50/30/20 and hybrid strategies shrink raw allocations when they exceed a cap; post-rounding reconciliation keeps each group at or below its cap. Needs, wants and savings rates may not sum to more than 1; recommendations require confirmation.

For a saving or purchase goal with remaining amount $R$ and monthly contribution $M > 0$, $n_s = \lceil R/M \rceil$. For current debt principal $L$ and nominal annual percentage rate $i$, the implementation uses monthly rate $r = i/(12 \times 100)$. With a debt deadline of $n_d$ months, the required end-of-month level payment is

$$P = \frac{Lr}{1 - (1 + r)^{-n_d}} \tag{3.8}$$

for $r > 0$, and $P = L/n_d$ for $r = 0$. A payment less than or equal to first-month interest triggers a negative-amortization warning; a zero payment is reported separately. Without a deadline, the implementation simulates month by month up to a 600-month default horizon. What-if currently reruns the saving/purchase function with an additional contribution; debt plans are not changed by that scenario, and no what-if result is persisted.

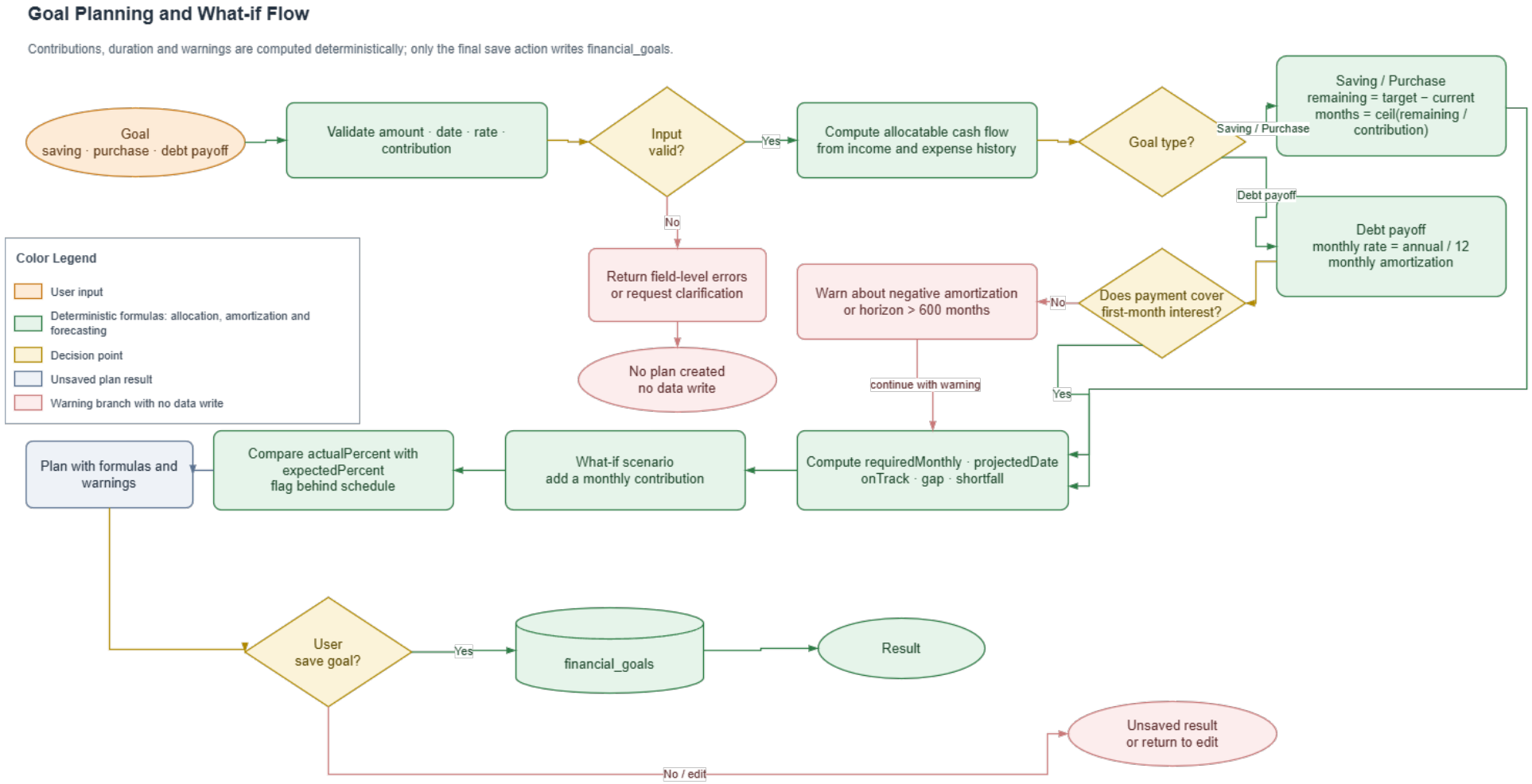


**Figure 3.9:** Goal planning and what-if flow; only final confirmation persists a goal.

#### 3.2.3.4. Grounded Insight Generation

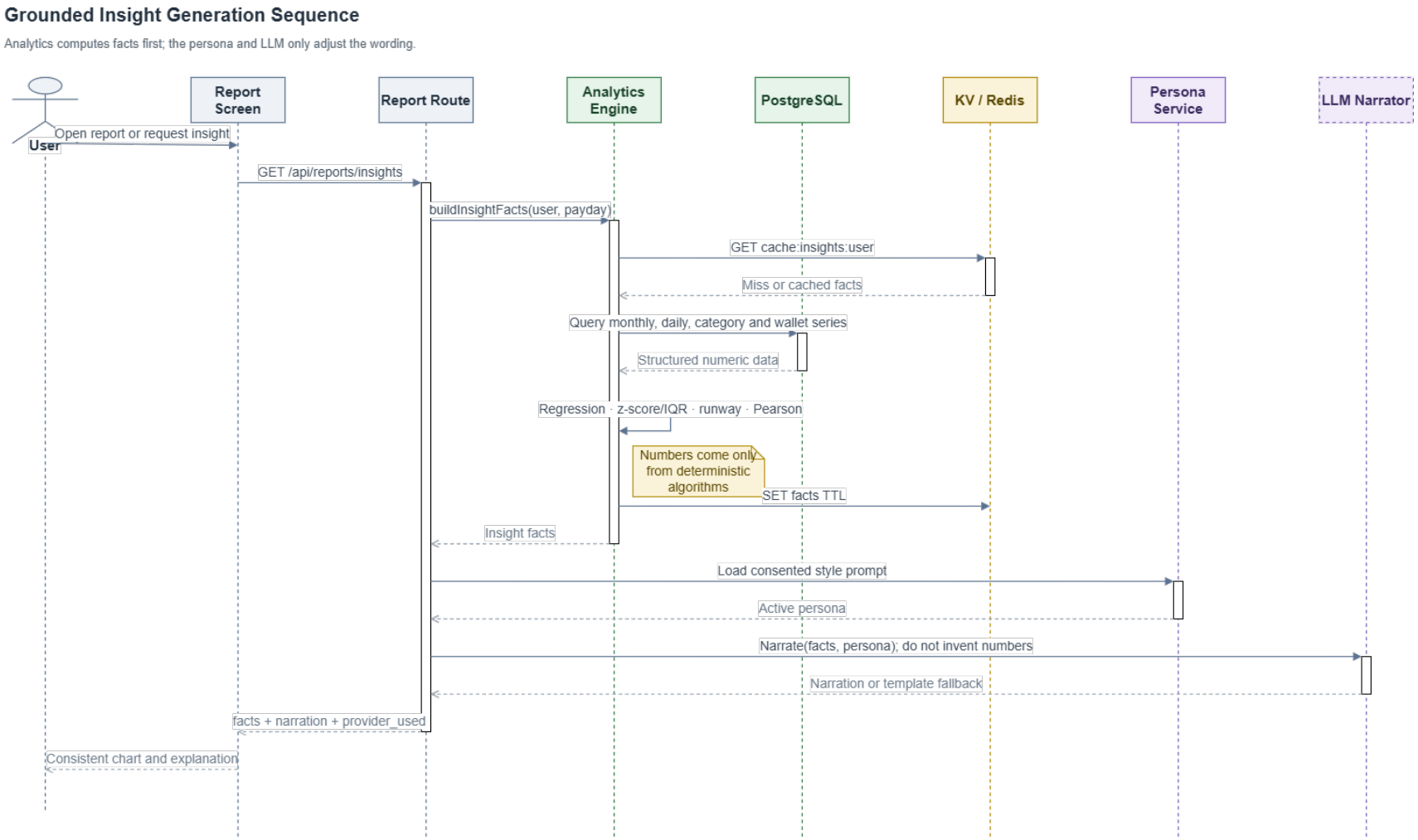


**Figure 3.10:** Grounded-insight sequence; Analytics returns facts before persona/LLM narration.

Facts contract version 1.0 returns `schema_version`, `generated_at`, each component value and `metadata.components`. Per-component metadata contains `unit`, `window`, `sample_count`, `method`, `parameters`, `warnings`, and `status` in `ok/no_signal/degraded`. The window object records `size`, `unit` and `zero_filled`; selected components add `category_count` or `wallet_count`. A local failure appears in `degraded_components` without removing the contract. A persona changes tone only with consent; the response includes narration and facts for UI/test comparison. A numeric checker covering every expression remains a target design.

The calculation path, numeric examples and boundary cases for matching, correction, Analytics, Budget, Goal and the facts contract are expanded in Appendix C–C.6; this chapter keeps the main formulas and design decisions for a coherent core narrative.

### 3.2.4. UI Design

The interface follows a mobile-first design with five primary tabs: Dashboard, Budget, Chat, Report and More; Chat occupies the central position as the preferred input action. Shared design tokens use blue for the brand, slate for text and neutral surfaces for content. Green and red are reserved for the semantic distinction between income and expense. Headers, cards, list rows, buttons and spacing reuse the same tokens, while primary touch targets provide an effective minimum size of 44 pixels.

The images in this subsection were regenerated on 13 August 2026 from the current Expo web snapshot at a $390 \times 844$ CSS-pixel viewport, device scale factor 2 and synthetic demo data. Chat history is isolated so that the preview state remains legible. The images illustrate design decisions; they are not manual-test results, native-device screenshots or UAT evidence.

Figure 3.11 shows the Dashboard information hierarchy and the review-before-write interaction in Chat. The Dashboard prioritizes balance, income and expense, and recent transactions. Chat converts natural input into a card containing amount, category and date, followed by explicit confirm, edit and cancel actions. Displaying the card alone does not write to the ledger.

Figure 3.12 applies the same card language and semantic colours to two read-oriented flows. Budget organizes period progress, forecast warnings and recommendations into a visible hierarchy. Report separates income and expense totals, deficit, multi-period insight and category breakdown so that readers can move from summary to detail.

Figure 3.13 shows preview-before-save for goals: the backend validates the displayed values before enabling save. What-if guidance leaves the goal unchanged.

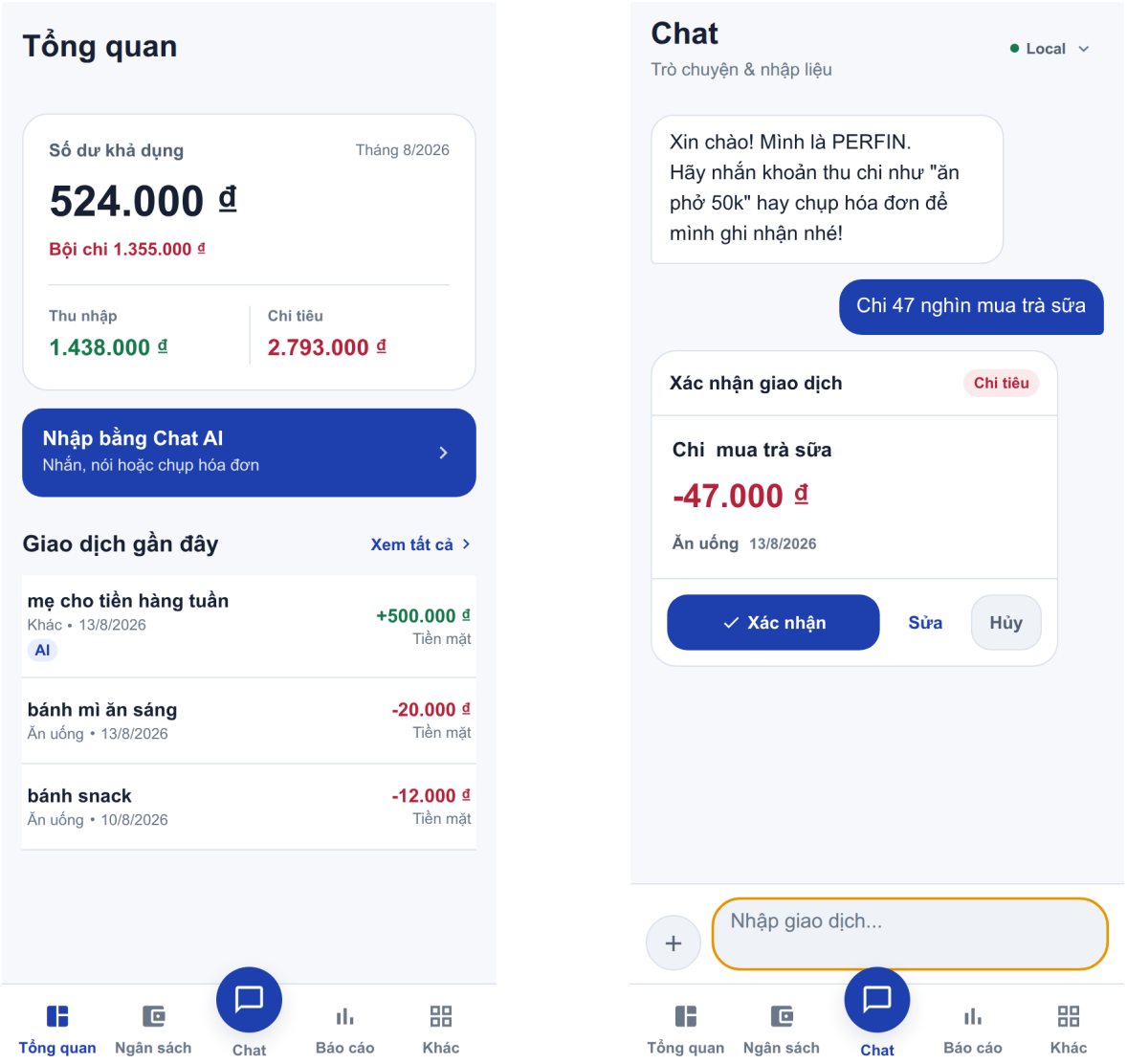


**Figure 3.11:** Dashboard (left) and a Chat transaction preview (right) in the redesigned interface.

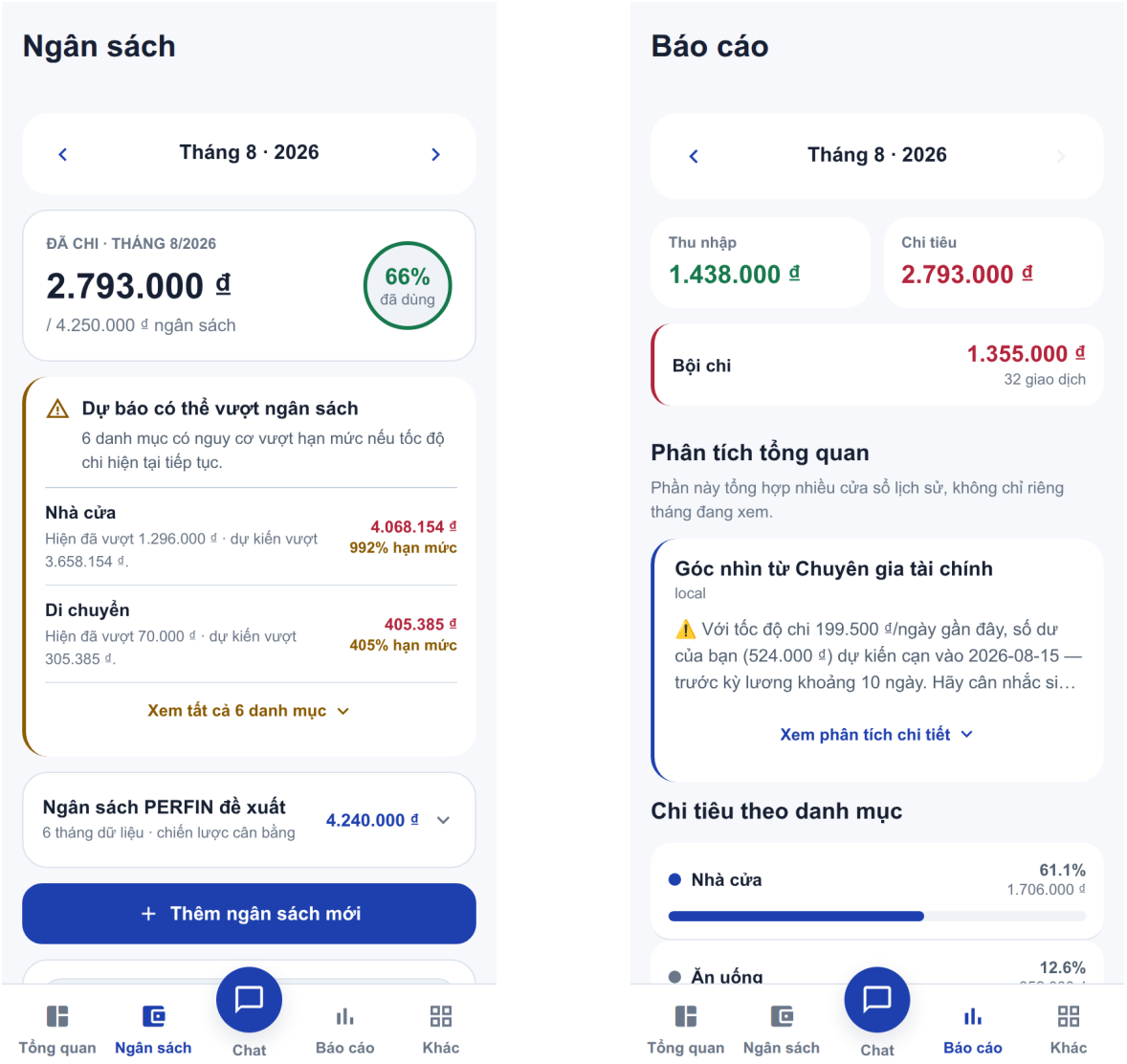


**Figure 3.12:** Budget (left) and Report (right) screens with consistent financial-data presentation.

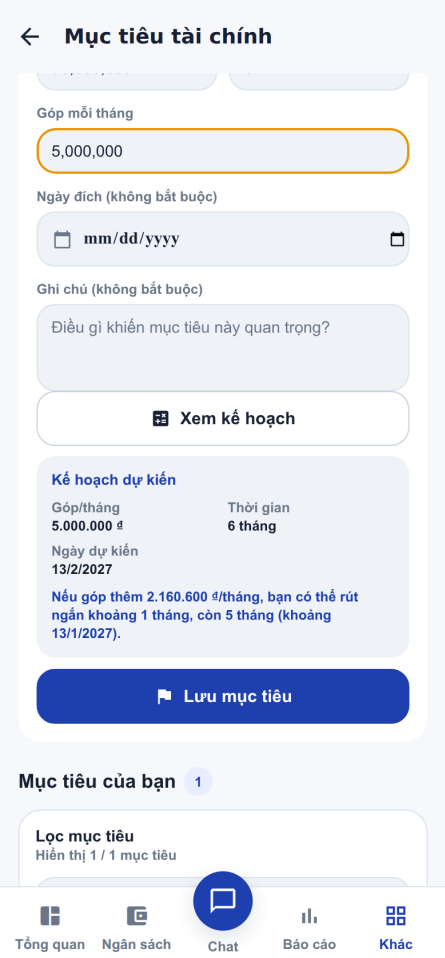


**Figure 3.13:** Goal-plan preview in the redesigned interface; the plan has not been saved at capture time.

## 3.3. TESTING

### 3.3.1. Test Plan

Testing is deliberately bounded to the correctness of data handling, algorithms and the prototype's main functional flows. The existing automated suite remains unchanged as a regression safety net, while manual system testing adds the author's interaction perspective. Testing conducted by the author is not treated as UAT.

**Table 3.16:** Simplified hybrid test strategy

| Level | Scope | Method | Scope of conclusion |
|---|---|---|---|
| Automated unit and regression tests | Formulas, invariants, parser and validation. | Run deterministic `node:test` cases over controlled fixtures. | Confirms specified cases only; it does not establish out-of-sample accuracy. |
| Internal integration | Route, service and model layers. | Check contracts, error branches and rollback with mocks or in-memory substitutes. | Confirms in-process coordination; it is not evidence of live PostgreSQL or Redis execution. |
| Manual system testing | Main functional flows in the interface. | The author uses demo data and compares observations with expected results. | Confirms selected scenarios only; it does not measure usability, OCR/STT accuracy or performance. |
| UAT | Target users. | Not conducted within this project. | It is distinct from the author's manual tests; no user-acceptance result is claimed. |

### 3.3.2. Results and Analysis

The two commands in Table 3.17 were rerun on 13 August 2026 against the current source snapshot using Node.js v24.16.0, with Redis and jobs disabled and the local parser selected. Results are reported in the unit produced by each command.

**Table 3.17:** Automated test results

| Command | Result | Scope of conclusion |
|---|---|---|
| `npm test` | 46/46 test files pass. | Covers backend unit/regression tests and internal integration with mocks or in-memory substitutes; live PostgreSQL and Redis are not confirmed. |
| `npm run test:ai` | 31/31 local-parser cases pass. | Confirms the designed fixture sentences only; it is not out-of-sample accuracy and does not evaluate Gemini, OCR or STT. |

**Status note:** The "Pass" values marked with an asterisk in Table 3.18 are draft assumptions, not verified results. Before submission, the author must execute every case, update its status from the observed result, and only then remove this note. Manual testing by the author is not UAT and does not establish OCR/STT accuracy, usability or performance.

**Table 3.18:** Manual functional test cases

| ID | Scenario/action | Expected result | Status |
|---|---|---|---|
| `TC-M01` | Create, edit, delete and restore a transaction through the form. | The transaction list and wallet balance reflect each operation correctly. | **Pass*** |
| `TC-M02` | Enter a complete transaction through chat, review its preview and confirm it. | Exactly one record is created, and only after confirmation. | **Pass*** |
| `TC-M03` | Enter incomplete or ambiguous content, then cancel. | The system requests missing information and writes no transaction. | **Pass*** |
| `TC-M04` | Submit a receipt image and a voice recording in turn; review the raw text. | Raw text can be edited or confirmed, and no data is saved automatically before confirmation. | **Pass*** |
| `TC-M05` | Open Dashboard/Report, reconcile income and expense totals, and export CSV. | Totals match the demo data; the CSV downloads and opens successfully. | **Pass*** |

| ID | Scenario/action | Expected result | Status |
|---|---|---|---|
| TC-M06 | Create a budget for a period containing transactions. | Spent and remaining amounts match transactions in the period. | **Pass*** |
| TC-M07 | Create a goal and run a what-if scenario. | The simulation is displayed without automatically changing the original plan. | **Pass*** |
| TC-M08 | Create a recurring bill and record one payment. | Status is updated and no duplicate transaction is created. | **Pass*** |

The automated results apply only to the source snapshot and fixtures used in this run. Tests using mocks or in-memory substitutes do not establish live database, queue or worker behavior. The eight manual cases cover selected flows only after author verification; they do not replace UAT or support claims about usability, media accuracy, latency or production load.

# CHAPTER 4: CONCLUSION

## 4.1. OBJECTIVE COMPLETION

PERFIN now forms a prototype with a clear separation between data, algorithms and the LLM layer. It contains 18 business tables plus the technical `_migrations` table, transaction, analytics, feedback, budget, goal and state modules, and a hybrid automated/manual test strategy. Its central contribution is not replacing business logic with an LLM, but using the model as a language-conversion layer while the backend retains validation, computation and data-write authority.

**Table 4.1:** Objectives and evidence

| Code | Main result | Conclusion |
|---|---|---|
| O1 | Migrations, ERD, constraints and tested transactional flows. | Prototype-complete; broader DB fault injection is needed. |
| O2 | Text/media adapters, typed draft, clarification, preview and confirmation. | Flow executes; OCR/STT accuracy is unmeasured. |
| O3 | Trend, anomaly, runway, recurring, correlation, budget and goal planner. | Implemented with unit tests; thresholds need representative calibration. |
| O4 | Tool schema, service validation, facts–narrator flow and fallback. | Boundary implemented; complete numeric faithfulness is unmeasured. |
| O5 | The hybrid plan includes 46/46 passing backend test files, 31/31 passing local-parser cases and eight specified manual functional cases. | Automated evidence is measured; manual statuses are draft assumptions awaiting verification, while UAT and live services remain unmeasured. |

The current data profile reports 5,328 source rows, zero validation errors and SHA-256 `418a9439...fbaed7`. Test results apply only to the source snapshot, fixtures and configuration disclosed in Section 3.3.

**Table 4.2:** Research questions and answer status

| RQ | Answer supported by the prototype | Boundary of the answer |
|---|---|---|
| RQ1 | Typed drafts, local media extraction, validation, preview and confirmation keep inferred values out of the ledger until review. | OCR/STT CER/WER and field accuracy require a controlled ground-truth set. |
| RQ2 | Deterministic OLS, robust anomaly signals, calendar-aware cashflow, budgets, goals and guarded correlation produce explainable facts; invariant tests are present. | Thresholds still require calibration on representative data. |
| RQ3 | Gemini is limited to language extraction/narration; backend services own validation, computation and writes. | Comparative provider benefit has not been evaluated; current evidence supports only the responsibility boundary. |

## 4.2. DELIVERABLES

Deliverables include mobile/backend source, PostgreSQL migrations, Redis/BullMQ configuration, Gemini plus local PaddleOCR/PhoWhisper adapters, the automated suite, a manual checklist, a reconciled demo dataset and this LaTeX report. The prototype supports text/media entry with review, ledger management, analytics, budgets and goals, and returns facts with controlled narration.

The architecture makes an LLM useful without granting it ledger authority; schema, validation, preview/confirmation and fact grounding remain mandatory controls. PERFIN is therefore a financial-data system with an LLM interaction layer, not a chatbot that calculates or decides on the user's behalf.

## 4.3. LIMITATIONS

1. The prototype uses `default_user` and lacks production authentication and authorization.
2. The controlled media set and reference transcripts have not yet been supplied; until then CER, WER and transaction-field accuracy are unmeasured.
3. Live Redis worker behaviour, retry/idempotency, p95 and UAT were not confirmed in a version-locked environment.

4. Fuzzy, anomaly, recurring and correlation thresholds and time windows are initial configurations rather than universal optima.
5. The eight manual-test statuses in the draft are not completion evidence until the author runs and verifies every case.
6. The numeric-grounding checker remains target design, and persona effects have not been evaluated.
7. Remaining gaps include true PDF export, full backup/restore, push notification and complete fresh-install wallet creation.

## 4.4. FUTURE WORK

Next priorities are to run and retain evidence for the eight manual cases; exercise rollback and duplicate prevention end-to-end with live PostgreSQL and Redis; build controlled media ground truth; implement numeric grounding and measure p50/p95 in a fixed environment; conduct UAT with target users; and add authentication, authorization, audit and secret management before real deployment.

Bank synchronization, shared wallets and high availability should follow only after data correctness and empirical evidence stabilize. This ordering preserves the basic-topic project's intended focus: data and algorithms are the core, while the LLM is a replaceable, measurable supporting component.

# BIBLIOGRAPHY


[1] A. Lusardi and O. S. Mitchell, "The Economic Importance of Financial Literacy: Theory and Evidence," *Journal of Economic Literature*, vol. 52, no. 1, pp. 5–44, 2014.

[2] V. I. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[3] L. R. Dice, "Measures of the Amount of Ecologic Association Between Species," *Ecology*, vol. 26, no. 3, pp. 297–302, 1945.

[4] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*, 5th ed. Hoboken, NJ, USA: Wiley, 2012.

[5] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[6] K. Pearson, "Note on Regression and Inheritance in the Case of Two Parents," *Proceedings of the Royal Society of London*, vol. 58, pp. 240–242, 1895.

[7] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proc. 9th International Conference on Document Analysis and Recognition*, vol. 2, pp. 629–633, 2007.

[8] A. Radford *et al.*, "Robust Speech Recognition via Large-Scale Weak Supervision," *arXiv preprint arXiv:2212.04356*, 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[9] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[10] Gemini Team, "Gemini: A Family of Highly Capable Multimodal Models," *arXiv preprint arXiv:2312.11805*, 2023.

[11] Google AI for Developers, "Function calling with the Gemini API," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs/function-calling. [Accessed: 15-Jul-2026].

[12] PostgreSQL Global Development Group, "PostgreSQL Documentation — Transactions and Data Definition," 2026. [Online]. Available: https://www.post

gresql.org/docs/current/tutorial-transactions.html and https://www.postgresql.org/docs/current/ddl.html. [Accessed: 15-Jul-2026].

[13] Redis Ltd., "EXPIRE," *Redis Commands Documentation*, 2026. [Online]. Available: https://redis.io/docs/latest/commands/expire/. [Accessed: 15-Jul-2026].

[14] Taskforce.sh, "BullMQ Documentation: Idempotent Jobs and Retrying Failing Jobs," 2026. [Online]. Available: https://docs.bullmq.io/patterns/idempotent-jobs and https://docs.bullmq.io/guide/retrying-failing-jobs. [Accessed: 15-Jul-2026].

[15] Money Lover, "Money Lover — Money Manager, Budget Expense Tracker," 2026. [Online]. Available: https://moneylover.me. [Accessed: 24-Jul-2026].

[16] MISA JSC, "MISA MoneyKeeper — Personal Expense Management Application," 2026. [Online]. Available: https://www.misa.vn/moneykeeper. [Accessed: 24-Jul-2026].

[17] X. Du *et al.*, "PP-OCR: A Practical Ultra Lightweight OCR System," *arXiv preprint arXiv:2009.09941*, 2020. [Online]. Available: https://arxiv.org/abs/2009.09941

[18] H. T. Nguyen *et al.*, "PhoWhisper: Automatic Speech Recognition for Vietnamese," *arXiv preprint arXiv:2406.02555*, 2024. [Online]. Available: https://arxiv.org/abs/2406.02555

[19] S. Amershi *et al.*, "Guidelines for Human–AI Interaction," in *Proc. CHI*, 2019, pp. 1–13, doi:10.1145/3290605.3300233.

[20] J. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, 2023, doi:10.1145/3571730.

[21] T. Schick *et al.*, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[22] M. Drobac, "Evaluation of OCR Accuracy," *International Journal on Document Analysis and Recognition*, 2020, doi:10.1007/s10032-020-00359-9.

[23] National Institute of Standards and Technology, "Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management," NIST CSWP 01162020, 2020, doi:10.6028/NIST.CSWP.01162020.

[24] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[25] D. R. Roberts *et al.*, “Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure,” *Ecography*, vol. 40, no. 8, pp. 913–929, 2017, doi:10.1111/ecog.02881.

[26] K. Claessen and J. Hughes, “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs,” in *Proc. ICFP*, 2000, pp. 268–279, doi:10.1145/351240.351266.

[27] T. Y. Chen *et al.*, “A Survey on Metamorphic Testing,” *IEEE Transactions on Software Engineering*, 2018, doi:10.1109/TSE.2018.2866449.

[28] ISO/IEC/IEEE, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE Std 29148:2018, 2nd ed., 2018.

# CHAPTER A: INSTALLATION GUIDE

The minimum environment is Node.js/npm, PostgreSQL and Expo Go or a browser. Redis 7 is recommended for the worker. Without Redis, the API may use an in-memory KV fallback during development, but queue and retry behaviour is not demonstrated.

## A.1. BACKEND SETUP

From the project root, copy the sample configuration, supply PostgreSQL settings and retain `AI_PROVIDER=local` when no Gemini key is used. Never commit secrets or service-account files.

```
cd demo/backend
cp .env.example .env
npm install
npm run migrate
npm start
```

The API defaults to http://127.0.0.1:3000. Use that URL for a basic availability check. Recreate schema and seed data only on a development/test database:

```
npm run migrate:fresh
```

## A.2. REDIS AND WORKER

From `demo`, start Redis with Docker Compose and open another terminal for the worker:

```
docker compose up -d redis
cd backend
npm run worker
```

Verify `REDIS_URL`, `JOBS_ENABLED` and timezone in `backend/.env`. `docker compose down` stops Redis while retaining its volume.

## A.3. FRONTEND SETUP

```
cd demo/frontend
npm install
EXPO_PUBLIC_API_URL=http://127.0.0.1:3000 npm start
```

For a phone on the same LAN, replace `127.0.0.1` with the backend host's LAN address and use `npm run start:lan`. If the device cannot reach that address, use Expo tunnel plus a separate backend tunnel. Never expose AI secrets through frontend environment variables.

# CHAPTER B: QUICK USER GUIDE

1. Open the application and check that Dashboard shows balance, total income/expense and recent transactions.
2. Add a transaction from Transactions, or open Chat and enter a sentence such as "ăn phở 50k sáng nay".
3. For image/audio, review and edit the OCR text or transcript. Media results are not stored automatically.
4. Review amount, type, date, category and wallet in the preview; edit, cancel or confirm. Only confirmation creates a transaction.
5. Use Budget for limits, Reports for trends/insights and Goals for plans and what-if scenarios.
6. Correct an incorrect category. Confirmed corrections may be retrieved for a later input.

The prototype has no login and uses `default_user`. Do not enter sensitive data or treat results as guaranteed investment advice. The interface and goal-plan preview are illustrated together in Section 3.2.4.

## B.1. SUPPLEMENTARY FUNCTIONAL REQUIREMENTS

The following full tables retain the same fields and identifiers as the specification template. They are moved here only to keep the main chapter focused on the nine core requirements (FR-01–05 and FR-07–10); the summary table in Chapter 3 still lists all twelve requirements.

**Table B.1:** FR-06 — Transfer Funds and Record Special Cash Flows

| **Item** | **Description** |
|---|---|
| **Feature name** | Wallet transfer and investment gain/loss. |
| **ID** | FR-06. |
| **User** | User. |
| **Necessity** | Desirable. |
| **Classification** | Complex. |
| **Participants and concerns** | One user action must update both wallets and history consistently. |
| **Summary** | Record internal transfers without changing total assets and classify investment cash flow correctly. |

| Item | Description |
|---|---|
| **Relationships** | Uses FR-05 wallet and transaction entities. |
| **Normal flow** | **Step 1:** Enter source, target, amount and date.<br>**Step 2:** Validate distinct wallets.<br>**Step 3:** Lock in stable order.<br>**Sub 1:** Debit, credit and write history.<br>**Step 4:** Commit and return the result. |
| **Subflows** | **Sub 1 — Transaction:** Subtract from the source.<br>**Sub 1.1:** Add to the target.<br>**Sub 1.2:** Write `wallet_transfers`; the net change equals zero. |
| **Alternate flows** | **Invalid wallets:** Same, missing, out-of-scope or different-currency wallets roll back.<br>**Invalid amount or date:** Roll back the transaction.<br>**Negative source balance:** Do not reject solely because the resulting source balance is negative. |

**Table B.2:** FR-11 — Manage Recurring Bills and Proactive Jobs

| Item | Description |
|---|---|
| **Feature name** | Recurring bills, payments and proactive messages. |
| **ID** | FR-11. |
| **User** | User; Background worker. |
| **Necessity** | Desirable. |
| **Classification** | Complex. |
| **Participants and concerns** | The user needs correct due dates; the worker needs retry and idempotency; PostgreSQL must prevent duplicate effects. |
| **Summary** | Manage schedules, record one payment per period and generate grounded reminders/alerts. |
| **Relationships** | Reads FR-05/FR-07; may call FR-08 for wording only. |

| Item | Description |
|---|---|
| **Normal flow** | **Step 1:** Create a valid schedule.<br>**Step 2:** The scheduler enqueues a user-scoped job.<br>**Step 3:** The worker loads due data and computes deterministic facts.<br>**Sub 1:** Invoke the idempotent handler.<br>**Step 4:** Persist an event by unique key and advance the schedule after success. |
| **Subflows** | **Sub 1 — Payment:** Write payment history.<br>**Sub 1.1:** Write the transaction.<br>**Sub 1.2:** Update the next due date atomically. |
| **Alternate flows** | **Duplicate event or job:** Return the existing result.<br>**Transient failure:** Retry with backoff.<br>**Redis failure:** Stop the worker while the interactive API may continue. |

**Table B.3:** FR-12 — Export Data and Remove Expired Files

| Item | Description |
|---|---|
| **Feature name** | Data export, export history and expired-file cleanup. |
| **ID** | FR-12. |
| **User** | User; Background worker. |
| **Necessity** | Optional. |
| **Classification** | Medium. |
| **Participants and concerns** | The user needs correct data; the exporter needs safe escaping; cleanup cannot remove valid files. |
| **Summary** | Generate filtered CSV, store history and delete files after TTL. |
| **Relationships** | Reads FR-05, may use FR-03 from chat, and delegates cleanup to the worker. |
| **Normal flow** | **Step 1:** Select the format and period.<br>**Step 2:** Validate and query profile data.<br>**Sub 1:** Create a safe file.<br>**Step 3:** Write `export_history`.<br>**Step 4:** Return a download link. |

| Item | Description |
| --- | --- |
| **Subflows** | **Sub 1 — CSV:** Escape formula-like cells.<br>**Sub 1.1:** Write a UTF-8 BOM and row count.<br>**Sub 1.2:** Assign the TTL. |
| **Alternate flows** | **Empty data:** Return a header-only file.<br>**PDF format:** The path once labelled PDF only creates HTML and is not claimed as a true PDF export. |

# CHAPTER C: EXTENDED ALGORITHM SPECIFICATIONS

This appendix expands the formulas and rules summarized in Chapter 3. It lets a reader follow the path from scoped input data to facts or a plan; it does not introduce a second source of truth or change prototype behavior. Examples use synthetic values, while names, thresholds and statuses are checked against the current implementation.

## C.1. DATA AND FIXTURE DESCRIPTION

This section documents data used for profiling/import and controlled fixtures used by automated tests. The transaction CSV belongs to one demo profile rather than an independent evaluation set. The report publishes aggregate statistics and checksums only; raw transaction descriptions are not copied into the appendix.

### *C.1.1. Dataset inventory*

**Table C.1:** Data sources, fixtures and roles

| Source | Size/scope | Role and status |
|---|---|---|
| Transaction CSV | 5,328 source rows, 8 columns; 1 Jan 2022–10 Aug 2026. | `dataFinance.csv` is used for import and data profiling. Dry-run validation rejects zero rows; SHA-256 is `418a9439...fbaed7`. |
| Category mapping | Crosswalk from the historical taxonomy to PERFIN's 17 type/category labels. | `dataFinance.category-map.json` is checked against the current CSV checksum; its SHA-256 is `8072dc3d...c6b573c`. It creates post-import gold labels, not parser predictions. |
| Media fixtures | Two images and one audio file for internal use; the images contain PII and are unsafe to distribute. | `media-fixtures.json` supports input/preview-flow checks, not OCR/STT accuracy claims. |

| Source | Size/scope | Role and status |
|---|---|---|
| Analytics/unit fixtures | Synthetic or anonymized series with known slope, outlier, cadence and correlation. | Used to check Analytics/Budget/Goal formulas, edge cases and invariants; not representative of real users. |

### C.1.2. Raw schema and category crosswalk

The CSV must contain the following eight columns in this exact order. The `planFinanceCsvImport` function validates the schema, dates, amounts, direction and mapping checksum before creating normalized records.

**Table C.2:** CSV schema and post-import fields

| CSV column | Valid values | Transformation and meaning |
|---|---|---|
| `Title` | Non-empty string, at most 200 characters. | Free-text transaction description; whitespace is normalized into `description` while the original is retained in `original_text`. |
| `Budget` | Non-zero number; positive for income and negative for expense. | Absolute value becomes `amount`; it must equal the absolute value of `Cost`. |
| `Cost` | Positive VND string, optionally containing ₫. | Parsed as a positive integer and cross-checked against `Budget`; it is not a second money measure. |
| `Date` | Existing date in `DD/MM/YYYY` format. | Converted to ISO `transaction_date` in `YYYY-MM-DD` form. |
| `Ex/In` | `Expenses` or `In-come`. | Maps to `type=expense` or `type=income`; other values are rejected. |
| `Special` | Optional string; may be blank. | Stored as `special` metadata; a blank cell becomes `null`. |

| CSV column | Valid values | Transformation and meaning |
|---|---|---|
| `Type Expenses` | Historical expense taxonomy; populated only when `Ex/In` is `Expenses`. | Looked up in the crosswalk to create `categoryName`; one expense row without a source label is intentionally mapped to *Other*. |
| `Type In come` | Historical income taxonomy; populated only when `Ex/In` is `In-come`. | Looked up in the crosswalk to create `categoryName`; the opposite-direction category column must be blank. |

The importer does not silently remove duplicates: the current dry-run reports 23 exact-duplicate groups and 24 extra rows, but the source has no ID or timestamp sufficient to call them errors, so the default mode retains them. The current CSV checksum prefix is `418a...`; changing one byte stops the import instead of applying an old mapping to new data.

### C.1.3. Size, distribution and data quality

**Table C.3:** Importer quality summary for the transaction set

| Attribute | Value | Interpretation |
|---|---|---|
| Source rows / validation rejects | 5,328 / 0 | Current dry-run profile; database apply remains a separate measured step. |
| Date range | 1 Jan 2022–10 Aug 2026 | The profile covers the current data snapshot. |
| Amount totals | Income 373,432,659 VND; expense 373,062,659 VND | Net cashflow is 370,000 VND; the values cross-check importer and report output. |
| Amount outlier flag | 615 rows above the IQR threshold 91,500 VND | Outliers are retained because they can be legitimate tuition, lending or household payments. |
| Exact duplicates | 23 groups / 24 extra rows | Retained by default to preserve the source; no duplicate error is asserted without a transaction identifier. |

### C.1.4. Fixture scope and limitations

Analytics/Budget/Goal fixtures set known slope, zero-fill, outlier, cadence, correlation, budget caps and what-if outcomes for deterministic comparison. The CSV represents one demo profile, is primarily Vietnamese and uses VND; raw content is not treated as anonymized material for public release. Existing media fixtures remain private because their manifest marks PII, and they are insufficient as ground truth. These sources therefore support only the specified cases, not out-of-sample accuracy or production generalization.

## C.2. SCOPE AND DATA CONVENTIONS

The pure functions in Analytics, Budget, Goal and Feedback do not access the database or the LLM directly. Services scope the data to a profile, convert it to flat arrays or objects, call deterministic helpers, and attach metadata. The following conventions are shared:

**Table C.4:** Shared input conventions for the algorithms

| Convention | Application |
|---|---|
| Data scope | Every query receives `userId/userKey`; records outside the profile do not enter a calculation. |
| Currency | Runway uses VND and sums only `cash`, `bank` and `e_wallet`; it never converts foreign currency. |
| Ledger | Soft-deleted transactions are excluded from reports; a negative balance remains valid. |
| Calendar axis | Days/months/weeks are generated for the complete fixed window; inactive periods are zero-filled, while an incomplete current month or week is excluded from comparisons. |
| Missing data | A function returns `null`, an empty list or `no_signal`; it does not replace missing evidence with an invented number. |
| Rounding | Plan amounts are rounded at output; group-cap invariants are checked again after rounding. |
| Writes | Preview, correction, recommendation and what-if are calculated results; persistence still requires the domain service and confirmation. |

Default windows are 14 days for runway, 30 days for anomaly, 60 days for day-of-week analysis, 200 days for recurring discovery, six completed months for trend and 12 completed ISO weeks for correlation. The facts contract uses version `1.0`; a failure in one component does not discard the others.

## C.3. CLASSIFICATION AND CORRECTION RETRIEVAL

### *C.3.1. Normalization and category scoring*

`normalizeForMatch` applies Unicode NFD, removes diacritics, maps `đ/Đ` to `d/D`, lowercases, replaces non-alphanumeric runs with spaces, and collapses repeated spaces. Thus " Điện-thoại! " becomes `dien thoai`. Exact, alias and fuzzy comparisons all use the normalized string and consider only the same `type` when the caller provides a transaction type.

For two normalized strings, the matcher computes three scores:

1. **Edit score:** $s_{edit} = 1 - d_{lev}(u, v)/\max(|u|, |v|)$, where $d_{lev}$ is Levenshtein distance.
2. **Dice score:** twice the number of overlapping tokens divided by the number of distinct tokens in the two token sets, then multiplied by $0.92$.

3. **Containment score:** when the smaller token set is contained in the larger and has at least two tokens, use $\min(0.94, 0.82 + 0.12|U|/|V|)$; otherwise use 0.

The final score is

$$s = \max(s_{edit},\ 0.92 s_{dice},\ s_{contain}). \tag{C.1}$$

The decision pipeline is:

1. Scope categories by transaction type and select the "Khác" fallback (or the first category if that fallback is absent).
2. If a normalized category name equals the input, return `exact` with confidence 1.
3. If exactly one alias equals the input, return `alias_exact` with confidence 0.98; aliases resolving to multiple categories return the fallback with reason `ambiguous_alias`.
4. Rank fuzzy scores over each category name and its aliases. Inputs longer than four characters require $s \geq 0.82$; inputs of at most four characters require $s \geq 0.90$. If a runner-up exists, the score gap must be at least $0.08$.
5. If the threshold or margin is not met, return the fallback with reason `below_threshold` or `ambiguous_fuzzy`; do not silently choose the nearest candidate.

For "ăn uống", normalization gives `an uog`; relative to `an uong`, Levenshtein distance is 1 over length 7, so $s_{edit} = 6/7 \approx 0.857$. The score exceeds the 0.82 threshold and, without a competing candidate, returns "Ăn uống" with `matchKind=fuzzy`. In contrast, the alias "điện thoại" points to both "Điện tử" and "Hóa đơn & Dịch vụ" and must return "Khác" for clarification.

Phrase alias lookup also requires word boundaries. When several aliases match, the alias with more tokens wins; a tie in token count and length across different categories returns `ambiguous_alias`. This prevents a short alias such as "an" from matching inside "quần áo".

### C.3.2. Correction ranking

A correction is useful only when a log has `feedback_type=classification`, original text and a corrected category different from the original category. The label key prioritizes `type + normalized category name`; an id is used only when the name is absent. The service loads at most 200 recent candidates from the current profile.

For every candidate, the service computes `textSimilarity`, keeps an exact match with score 1 or a fuzzy match with a minimum score of 0.88, and removes

a different income/expense type. Candidates are grouped by corrected label:

$$w_k = \sum_{j \in k} s_j, \qquad agreement = \frac{\max_k w_k}{\sum_k w_k}. \tag{C.2}$$

The winning candidate exposes `support` (group count), `bestScore` (maximum score) and confidence rounded from $s_{best} \times agreement$. Retrieval is rejected when:

- the input is empty or no candidate reaches the threshold;
- identical normalized text has multiple corrected labels;
- the non-exact winner is below 0.88;
- a competing label leaves agreement below 0.8;
- a legacy log lacks `type` while the caller requests a specific type.

Few-shot examples use a separate policy: at most five by default, an absolute limit of 20, and a minimum score of 0.35. Conflicting labels for the same typed normalized input are discarded, and duplicate labels keep only the nearest example. Correction therefore is not majority voting and cannot change transaction type.

Normalization, typo, ambiguous alias, exact/fuzzy correction, label-conflict and name-only/id+name cases use regression fixtures in `feedback.test.js`; results apply only to the specified cases.

## C.4. FINANCIAL ANALYSIS ALGORITHMS

### *C.4.1. Calendar axes and runway*

The helpers create an axis first, initialize totals to zero, and then add rows that fall within that axis:

- `completeDailyTotals` completes the day axis;
- `completeMonthlyByCategory` completes each category's month axis;
- `completeWeeklyByCategory` completes the completed ISO-week axis.

Correlation uses completed ISO weeks; the week containing today is excluded. This preserves the denominator when a category is inactive during a period.

Runway first filters wallets to `currency=VND` and types `cash`, `bank` and `e_wallet`:

$$B = \sum_{w \in \text{eligible wallets}} balance_w. \tag{C.3}$$

For a 14-day window and daily expense $e_j$ (zero-spend days remain zero),

$$\bar{e}_W = \frac{1}{W}\sum_{j=1}^{W} e_j, \qquad d = \left\lfloor \frac{B}{\bar{e}_W} \right\rfloor. \tag{C.4}$$

If $B \le 0$, the result is $d = 0$ with `alreadyDepleted`; if $\bar{e}_W \le 0$, the result has `daysLeft=null` and no depletion date is invented. Payday is only a comparison warning for the next occurrence; day 31 is clamped to the last day of the target month.

For two liquid VND wallets totaling 300,000 VND and 14 days each spending 100,000 VND, $\bar{e}_W = 100,000$ and $d = 3$. USD and investment wallets do not increase $B$.

### C.4.2. Trend and anomaly

For a zero-filled series $y_0, \ldots, y_{n-1}$, OLS uses $x_i = i$, $\bar{x}$ and $\bar{y}$ to calculate

$$b = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad a = \bar{y} - b\bar{x}. \tag{C.5}$$

It computes the fit statistic $R^2 = 1 - SS_{res}/SS_{tot}$. The next-step forecast is $\max(0, \text{round}(bn + a))$. Money slope/intercept are rounded and $R^2$ is rounded to three decimals. The engine emits a trend only with at least three positive-spend months, $b > 0$, $R^2 \ge 0.5$ and an average stepwise percentage change of at least 10% over steps with a positive preceding value. A constant series has $R^2 = 0$ and produces no signal.

Anomaly detection uses the mean, sample standard deviation and interpolated quantiles over the complete 30-day axis:

$$z_i = \frac{e_i - \bar{e}}{s}, \qquad upperFence = Q_3 + 1.5(Q_3 - Q_1). \tag{C.6}$$

Only the upper tail is flagged when $z_i \ge 2.5$ or $e_i > upperFence$, and the engine requires at least four positive-spend days before calling the helper. When $s = 0$, $z_i = 0$; when $IQR = 0$, the IQR branch is disabled. In the synthetic sample $[100, 100, 100, 1000]$, $Q_1 = 100$, $Q_3 = 325$ and the upper fence is 662.5, so 1000 is flagged by IQR even though its z-score is below 2.5.

### C.4.3. Recurring and day-of-week analysis

Recurring discovery removes diacritics, lowercases, removes digits from descriptions and groups positive expense transactions by normalized description. A group needs at least three occurrences, every amount must be within 15% of the mean, and positive date gaps must have population coefficient of variation at most 0.12. All gaps must fit one cadence:

**Table C.5:** Cadences used by recurring discovery

| Cadence | Day gap | Monthly factor | Conversion example |
|---|---|---|---|
| Weekly | 5–9 | $52/12$ | Four weekly payments are annualized over 52 weeks. |
| Monthly | 26–35 | $1$ | The average payment is the monthly estimate. |
| Quarterly | 80–100 | $1/3$ | A 900,000 VND quarterly payment becomes 300,000 VND/month. |

A candidate older than its cadence maximum from `asOf` is inactive. The output keeps the latest description, occurrence count, average amount, cadence, `lastSeen`, `nextExpected` and `monthlyEstimate`; there is no default amount ceiling. Three 900,000 VND payments approximately one quarter apart therefore remain a quarterly recurring item estimated at 300,000 VND/month.

Day-of-week analysis uses spending per active day in the 60-day window; an inactive day is not treated as a zero-spend observation. It needs at least four observed weekdays and a peak-to-mean ratio of 1.8; otherwise it returns `null`.

### C.4.4. Correlation

The engine creates a shared axis of 12 completed ISO weeks, zero-fills each category and keeps categories observed in at least four weeks. Pearson uses the shorter series length but returns `null` for fewer than three paired observations or zero variance:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}. \tag{C.7}$$

The engine scans every pair, keeps $r \geq 0.6$ and returns the pair with the greatest $r$. For proportional series $[100, 200, 300, 400]$ and $[200, 400, 600, 800]$, $r = 1$; the result describes co-movement in the selected window and does not imply causation.

## C.5. BUDGET AND FINANCIAL-GOAL PLANNING

### C.5.1. History summary and budget recommendation

`summarizeHistory` normalizes periods to `YYYY-MM`, aggregates income by month and aggregates expense by category id (or normalized name). With `expectedPeriods`, every listed month remains in the denominator even if it has no transaction. Thus a category spending 3,000,000 VND in one month of a three-month window has `average_spend=1,000,000`.

Each category is assigned to `needs` or `wants`; the default needs set contains food, transport, health, education, housing, bills/services and groceries, while `groupOverrides` can replace the assignment. With buffer $q$ over $m$ months,

$$b_c = (1+q)\frac{1}{m}\sum_{j=1}^{m} s_{c,j}. \tag{C.8}$$

The progress forecast receives `spent`, `amount_limit` and the target period. For the current month, $d_e = \max(1, \text{today's day})$; for a historical month, $d_e$ is the full month length. If $D$ is the number of days in the month,

$$\widehat{S}_D = \text{round}\left(\frac{S}{d_e}D\right), \qquad dailyRate = \text{round}\left(\frac{S}{d_e}\right). \tag{C.9}$$

The result records:

- `daily_rate`, `projected_spend` and projected percentage;
- the `likely_to_exceed` flag and, when applicable, `projected_exceed_day`.

When the limit is positive and current spending is below it, the service returns an exceedance day only if it remains in the month; a zero limit does not create a fictitious day. For 2,000,000 VND spent on day 10 of a 30-day month with a 5,000,000-VND limit, the rate is 200,000 VND/day, the forecast is 6,000,000 VND and the projected exceedance day is 25. The corresponding cases are in `budget-forecast.test.js`.

The strategies behave differently:

- **category average:** uses $b_c$ directly.
- **50–30–20:** allocates group caps by each category's share of $b_c$, with default needs, wants and savings caps of $0.5I$, $0.3I$ and $0.2I$.
- **hybrid:** keeps $b_c$ when the group total is within its cap; otherwise it scales all group items by $cap/groupAverage$.

Each raw limit is rounded by `roundingIncrement`, 10,000 VND by default. `enforceRoundedCap` then repeatedly reduces one increment from a positive category with the largest overshoot until the group total is within its cap. A sum

needsRate+wantsRate+savingsRate above 1 is rejected; missing income falls back to category_average with a warning.

For history with 10,000,000 VND monthly income, 4,000,000 VND needs and 4,000,000 VND wants over three months, with zero buffer, 50–30–20 gives caps of 5,000,000 and 3,000,000 VND. Hybrid keeps needs at 4,000,000 but shrinks wants to 3,000,000. If two wants categories are 15,000 VND each, a 10,000-VND rounding increment and a 30,000-VND cap still produce a post-reconciliation total within the cap.

### C.5.2. Saving, debt payoff and what-if

For a saving/purchase goal, validation requires a positive target, non-negative current amount and a finite non-negative contribution. If contribution is null, the planner uses available surplus; an explicit user contribution of zero remains zero rather than being replaced by surplus. With remaining amount $R = \max(0, target - current)$ and monthly contribution $M > 0$,

$$n_s = \left\lceil \frac{R}{M} \right\rceil, \qquad projectedDate = addMonthsClamped(today, n_s). \tag{C.10}$$

End-of-month dates are clamped, so 31/01 plus one month becomes 28/02. If $R = 0$, the planner returns completed with horizon 0; if $M = 0$, horizon and projected date are null with NO_MONTHLY_CONTRIBUTION. A future deadline uses at least one contribution period in the same calendar month and reports requiredMonthly, gap and shortfall.

For current principal $L$, nominal annual percentage rate $i$ and $r = i/(12 \times 100)$, the level end-of-month payment over $n_d$ months is

$$P = \begin{cases} \dfrac{Lr}{1 - (1+r)^{-n_d}}, & r > 0, \\ \dfrac{L}{n_d}, & r = 0. \end{cases} \tag{C.11}$$

Without a deadline, the planner simulates each month: add interest, subtract a payment no larger than the amount due, and stop at zero balance or the horizon (600 months by default; validation permits at most 1200). A zero payment returns NO_MONTHLY_PAYMENT; a payment no larger than first-month interest returns NEGATIVE_AMORTIZATION; an unpaid balance after the horizon returns MAX_HORIZON_EXCEEDED. For $L = 10,000$, 12% annual interest and payment 1,000, the result is 11 months and 589.85 total interest; payment 100 is rejected because the balance does not decrease.

What-if accepts only saving or purchase plans, validates non-negative extra-

Monthly, and reruns the planner with original contribution plus the extra amount. It returns the new contribution, new horizon, months saved, feasibility transition and shortfall; it does not persist a goal or change a debt plan.

## C.6. FACTS CONTRACT AND LLM BOUNDARY

The Analytics Engine runs six independent components: trend, anomaly, runway, subscriptions, day-of-week and correlation. Each reads scoped data and returns a value or `null`, then the engine wraps it in one contract:

**Table C.6:** Fields in facts contract version 1.0

| Field | Meaning |
|---|---|
| `schema_version` | Contract shape version, currently `1.0`. |
| `generated_at` | ISO timestamp at which facts were generated. |
| Component values | Numeric values or result lists; no signal is represented by `null`. |
| `metadata.components` | Each component includes `unit`, `window`, `sample_count`, `method`, `parameters`, `warnings` and `status`. |
| `status` | `ok` when a result exists; `no_signal` when the contract is valid but evidence is insufficient; `degraded` when the component failed. |
| `degraded_components` | Failed component names; no internal error message or credential is exposed. |

The `window` object records size, unit and whether zero-fill was used; selected components add `category_count`, `wallet_count` or `observed_period_count`. Facts are cached for 600 seconds and reused only when schema, currency and payday context still match.

The numeric boundary is: SQL/Model reads source data, Analytics Engine computes facts, and only then does the narrator receive them. The LLM cannot call models, replace numbers, write transactions or turn `null` into a concrete value. A persona changes tone only when consent is enabled. A numeric grounding checker covering every numeric expression remains target design, not a measured result.

## CHAPTER D: REPRODUCTION COMMANDS

Run the two automated test commands from the backend directory with Redis/jobs disabled and the local parser selected. Record the commit, Node version and run time without storing secrets in artifacts.

```
cd demo/backend
export REDIS_ENABLED=false JOBS_ENABLED=false AI_PROVIDER=local
npm test
npm run test:ai
```

Current results are summarized in Section 3.3. These commands do not invoke live PostgreSQL/Redis, Gemini, OCR or STT and therefore provide no evidence for those services.

## CHAPTER E: DIAGRAM SOURCES

The report keeps 14 editable Draw.io sources and one overall use case. Every diagram label is English so the Vietnamese and English reports share the same set. Editable sources and manually exported PNG files are stored together in `latex/figures/drawio/`.

| No. | Subject | Basename |
|---|---|---|
| 1 | System context | `01-system-context` |
| 2 | Runtime architecture | `02-runtime-architecture` |
| 3 | Prototype deployment | `03-deployment` |
| 4 | Domain class diagram | `04-domain-class` |
| 5 | Physical ERD | `05-physical-erd` |
| 6 | LLM boundary | `06-llm-boundary` |
| 7 | Conversation state | `07-conversation-state` |
| 8 | Text transaction sequence | `08-text-sequence` |
| 9 | Multimodal input | `09-multimodal-flow` |
| 10 | Feedback and classification | `10-feedback-flow` |
| 11 | Grounded insight | `11-insight-sequence` |
| 12 | Goal planner and what-if | `12-goal-flow` |
| 13 | Background job | `13-worker-sequence` |
| 14 | Overall use case | `14-usecase-overview` |